

XCPC Templates

Skyclad Observer

Continue

October 6, 2026

Contents

1 代数结构 2

1.1 幺半群 2

1.2 幺半群作用 3

1.3 模数类 3

1.4 快速幂 4

2 图论 4

2.1 Base 4

2.2 最短路 4

2.2.1 Dijkstra 4

2.2.2 01-BFS 6

2.2.3 Bellman-Ford 6

2.2.4 SPFA 7

2.2.5 Floyd 7

2.2.6 路径重建 7

2.3 拓扑排序 8

2.4 欧拉通路/回路 8

2.5 最小生成树 9

2.5.1 Kruskal 9

2.5.2 Prim 10

2.5.3 Borůvka 10

2.5.4 Kruskal 重构树 11

2.6 连通性 12

2.6.1 强连通分量 12

2.6.2 边双连通分量 12

2.6.3 点双连通分量 13

2.6.4 圆方树 14

2.7 树链剖分 15

2.8 LCA 17

2.9 虚树 18

2.10 2-SAT 18

3 网络流 19

3.1 最大流最小割 19

3.2 最小费用最大流 20

4 数论 21

4.1 扩展欧几里得算法 21

4.2 线性筛 22

4.3 莫比乌斯反演 22

4.4 欧拉函数 23

4.5 Miller-Rabin 23

4.6 Pollard-Rho 24

4.7 约数构造 25

5 组合数学 25

5.1 组合数 25

5.2 康托展开 26

5.3 逆序对 26

5.4 容斥原理 27

5.5 zeta 变换 28

6 数据结构 28

6.1 线段树 28

6.1.1 线段树 28

6.1.2 懒标记线段树 30

6.1.3 动态开点线段树 32

6.2 树状数组 33

6.2.1 树状数组 33

6.2.2 区间加、单点查树状数组 34

6.2.3 区间加、区间和树状数组 34

6.2.4 二维树状数组 35

6.3 并查集 35

6.4 稀疏表 36

6.5 01-Trie 36

6.6 笛卡尔树 37

7 线性代数 38

7.1 矩阵 38

7.2 高斯消元 39

7.3 矩阵秩 39

7.4 逆矩阵 39

7.5 线性方程组 40

7.6 GF(2) 空间 40

7.6.1 线性基 40

7.6.2 逆矩阵 42

7.6.3 转置 42

8 字符串 42

8.1 Manacher 42

8.2 KMP 42

8.3 Trie 43

8.4 Z 函数 43

8.5 AC 自动机 43

9 杂项 44

9.1 debug 44

9.2 template 46

9.3 i128 46

9.4 日期 47

1 代数结构

1.1 么半群

```

1 // Template
2 // struct Monoid {
3 //     struct Type {
4 //         i64 v;
5 //     };
6 //     static Type e() {
7 //         return {};
8 //     }
9 //     static Type op(const Type& a, const Type& b) {
10 //         return {};
11 //     }
12 //     static Type from(const i64& x) {
13 //         return {x};
14 //     }
15 // };
16
17 template <typename T>
18 struct AddMono {
19     using Type = T;
20     static Type e() {
21         return {0};
22     }
23     static Type op(const Type& a, const Type& b) {
24         return {a + b};
25     }
26     static Type from(const i64& x) {
27         return {x};
28     }
29     static Type inv(const Type& a) {
30         return {-a};
31     }
32     static Type power(const Type& a, i64 k) {
33         return a * k;
34     }
35 };
36
37 template <typename T>
38 struct MulMono {
39     using Type = T;
40     static Type e() {

```

```

41         return {1};
42     }
43     static Type op(const Type& a, const Type& b) {
44         return {a * b};
45     }
46     static Type from(const i64& x) {
47         return {x};
48     }
49 };
50
51 template <typename T>
52 struct MinMono {
53     using Type = T;
54     static Type e() {
55         return {inf<T>};
56     }
57     static Type op(const Type& a, const Type& b) {
58         return {min(a, b)};
59     }
60     static Type from(const i64& x) {
61         return {x};
62     }
63 };
64
65 template <typename T>
66 struct MaxMono {
67     using Type = T;
68     static Type e() {
69         return {-inf<T>};
70     }
71     static Type op(const Type& a, const Type& b) {
72         return {max(a, b)};
73     }
74     static Type from(const i64& x) {
75         return {x};
76     }
77 };
78
79 template <typename T>
80 struct GcdMono {
81     using Type = T;
82     static Type e() {
83         return {0};

```

```

84     }
85     static Type op(const Type& a, const Type& b) {
86         return {gcd(a, b)};
87     }
88     static Type from(const i64& x) {
89         return {x};
90     }
91 };
92
93 template <typename T>
94 struct LcmMono {
95     using Type = T;
96     static Type e() {
97         return {1};
98     }
99     static Type op(const Type& a, const Type& b) {
100         return {lcm(a, b)};
101     }
102     static Type from(const i64& x) {
103         return {x};
104     }
105 };

```

1.2 幺半群作用

```

1 // Template
2 // struct ActMono {
3 //     using Mono = Mono;
4 //     using Tag = Tag;
5 //     using X = Mono::Type;
6 //     using Y = Tag::Type;
7 //     static X act(const X& x, const Y& y, int len) {
8 //         return {};
9 //     }
10 // };
11
12 template <typename T>
13 struct SumAdd {
14     using Mono = AddMono<T>;
15     using Tag = AddMono<T>;
16     using X = Mono::Type;
17     using Y = Tag::Type;
18     static X act(const X& x, const Y& y, int len) {
19         return {x + y * len};

```

```

20     }
21 };
22
23 template <typename T>
24 struct SumMul {
25     using Mono = AddMono<T>;
26     using Tag = MulMono<T>;
27     using X = Mono::Type;
28     using Y = Tag::Type;
29     static X act(const X& x, const Y& y, int len) {
30         return {x * y};
31     }
32 };
33
34 template <typename T>
35 struct MinAdd {
36     using Mono = MinMono<T>;
37     using Tag = AddMono<T>;
38     using X = Mono::Type;
39     using Y = Tag::Type;
40     static X act(const X& x, const Y& y, int len) {
41         return {x + y};
42     }
43 };
44
45 template <typename T>
46 struct MaxAdd {
47     using Mono = MaxMono<T>;
48     using Tag = AddMono<T>;
49     using X = Mono::Type;
50     using Y = Tag::Type;
51     static X act(const X& x, const Y& y, int len) {
52         return {x + y};
53     }
54 };

```

1.3 模数类

```

1 template <u32 P>
2 struct Mint {
3     u32 v;
4     Mint(i64 v_ = 0) : v((v_ % P) < 0 ? v_ + P : v_) {}
5     Mint inv() const {
6         i64 x, y;

```

```

7     exgcd(v, P, x, y);
8     return x;
9 }
10 Mint& operator+=(Mint rhs) & {
11     v += rhs.v;
12     if (v >= P) v -= P;
13     return *this;
14 }
15 Mint& operator-=(Mint rhs) & {
16     v -= rhs.v;
17     if (v >= P) v += P;
18     return *this;
19 }
20 Mint& operator*=(Mint rhs) & {
21     v = u64(v) * rhs.v % P;
22     return *this;
23 }
24 Mint& operator/=(Mint rhs) & {
25     return *this *= rhs.inv();
26 }
27 friend Mint operator-(Mint rhs) { return rhs.v ? Mint(P - rhs.v) : rhs; }
28 friend Mint operator+(Mint lhs, Mint rhs) { return lhs += rhs; }
29 friend Mint operator-(Mint lhs, Mint rhs) { return lhs -= rhs; }
30 friend Mint operator*(Mint lhs, Mint rhs) { return lhs *= rhs; }
31 friend Mint operator/(Mint lhs, Mint rhs) { return lhs /= rhs; }
32 friend bool operator==(Mint lhs, Mint rhs) { return lhs.v == rhs.v; }
33 friend bool operator!=(Mint lhs, Mint rhs) { return lhs.v != rhs.v; }
34 friend istream& operator>>(istream& is, Mint& v) { i64 x; is >> x; v = x;
return is; }
35 friend ostream& operator<<(ostream& os, const Mint& v) { return os << v.v; }
36 };
37 constexpr u32 P = 998244353;
38 using Fp = Mint<P>;

```

1.4 快速幂

```

1 i64 mul(i64 a, i64 b, i64 m) {
2     return static_cast<i128>(a) * b % m;
3 }
4
5 i64 power(i64 a, i64 b, i64 m) {
6     i64 res = 1 % m;
7     while (b > 0) {

```

```

8         if (b & 1) {
9             res = mul(res, a, m);
10        }
11        a = mul(a, a, m);
12        b >>= 1;
13    }
14    return res;
15 }

```

2 图论

2.1 Base

```

1 template <typename T>
2 struct Edge {
3     int u, v;
4     T w;
5     Edge() {}
6     Edge(int u_, int v_, T w_) : u(u_), v(v_), w(w_) {}
7 };
8
9 template <typename T>
10 struct Graph {
11     vector<Edge<T>> edges;
12     vector<vector<pair<int, T>>> adj;
13     int n;
14
15     Graph() : n(0) {}
16     Graph(int n_) : n(n_) {
17         adj.resize(n);
18     }
19
20     void addEdge(int u, int v, T w = {0}) {
21         edges.emplace_back(u, v, w);
22         adj[u].emplace_back(v, w);
23     }
24 };

```

2.2 最短路

2.2.1 Dijkstra

```

1 template <typename T>
2 pair<vector<T>, vector<int>> dijkstra(vector<vector<pair<int, T>>>& adj, int s) {
3     int n = adj.size();
4     vector<T> dist(n, inf<T>);

```

```

5  vector<int> pre(n, -1);
6  using P = pair<T, int>;
7  priority_queue<P, vector<P>, greater<P>> pq;
8
9  dist[s] = 0;
10 pq.emplace(0, s);
11
12 while (!pq.empty()) {
13     auto [d, u] = pq.top();
14     pq.pop();
15     if (d != dist[u]) {
16         continue;
17     }
18     for (auto& [v, w] : adj[u]) {
19         if (dist[v] > dist[u] + w) {
20             dist[v] = dist[u] + w;
21             pre[v] = u;
22             pq.emplace(dist[v], v);
23         }
24     }
25 }
26
27 return {dist, pre};
28 }
29
30 template <typename T>
31 pair<vector<T>, vector<int>> dijkstraDense(vector<vector<pair<int, T>>& adj, int
32 s) {
33     int n = adj.size();
34     vector<T> dist(n, inf<T>);
35     vector<int> pre(n, -1);
36     vector<int> done(n, false);
37
38     dist[s] = 0;
39
40     for (int _ = 0; _ < n; _++) {
41         int u = -1;
42         for (int v = 0; v < n; v++) {
43             if (!done[v] && (u == -1 || dist[v] < dist[u])) {
44                 u = v;
45             }
46         }
47         if (u == -1 || dist[u] == inf<T>) {
48             break;

```

```

48     }
49     done[u] = true;
50     for (auto [v, w] : adj[u]) {
51         if (dist[v] > dist[u] + w) {
52             dist[v] = dist[u] + w;
53             pre[v] = u;
54         }
55     }
56 }
57
58 return {dist, pre};
59 }
60
61 template <typename T>
62 tuple<vector<T>, vector<int>, vector<int>> dijkstra(vector<vector<pair<int, T>>&
63 adj, vector<int>& s) {
64     int n = adj.size();
65     vector<T> dist(n, inf<T>);
66     vector<int> pre(n, -1), root(n, -1);
67     using P = pair<T, int>;
68     priority_queue<P, vector<P>, greater<P>> pq;
69
70     for (auto& u : s) {
71         dist[u] = 0;
72         root[u] = u;
73         pq.emplace(0, u);
74     }
75
76     while (!pq.empty()) {
77         auto [d, u] = pq.top();
78         pq.pop();
79         if (d != dist[u]) {
80             continue;
81         }
82         for (auto& [v, w] : adj[u]) {
83             if (dist[v] > dist[u] + w) {
84                 dist[v] = dist[u] + w;
85                 pre[v] = u;
86                 root[v] = root[u];
87                 pq.emplace(dist[v], v);
88             }
89         }

```

```

90
91     return {dist, pre, root};
92 }

```

2.2.2 01-BFS

```

1 template <typename T>
2 pair<vector<T>, vector<int>> bfs_01(vector<vector<pair<int, T>>& adj, int s) {
3     int n = adj.size();
4     vector<T> dist(n, inf<T>);
5     vector<int> pre(n, -1);
6     deque<int> que;
7
8     dist[s] = 0;
9     que.push_front(s);
10
11     while (!que.empty()) {
12         auto u = que.front();
13         que.pop_front();
14         for (auto& [v, w] : adj[u]) {
15             if (dist[v] > dist[u] + w) {
16                 dist[v] = dist[u] + w;
17                 pre[v] = u;
18                 if (w == 0) {
19                     que.push_front(v);
20                 } else {
21                     que.push_back(v);
22                 }
23             }
24         }
25     }
26
27     return {dist, pre};
28 }
29
30 template <typename T>
31 tuple<vector<T>, vector<int>, vector<int>> bfs_01(vector<vector<pair<int, T>>&
adj, vector<int>& s) {
32     int n = adj.size();
33     vector<T> dist(n, inf<T>);
34     vector<int> pre(n, -1), root(n, -1);
35     deque<int> que;
36
37     for (auto& u : s) {
38         dist[u] = 0;

```

```

39     root[u] = u;
40     que.push_front(u);
41 }
42
43 while (!que.empty()) {
44     auto u = que.front();
45     que.pop_front();
46     for (auto& [v, w] : adj[u]) {
47         if (dist[v] > dist[u] + w) {
48             dist[v] = dist[u] + w;
49             pre[v] = u;
50             root[v] = root[u];
51             if (w == 0) {
52                 que.push_front(v);
53             } else {
54                 que.push_back(v);
55             }
56         }
57     }
58 }
59
60 return {dist, pre, root};
61 }

```

2.2.3 Bellman-Ford

```

1 template <typename T>
2 pair<vector<T>, vector<int>> bellmanFord(vector<vector<pair<int, T>>& adj, int s)
{
3     int n = adj.size();
4     vector<T> dist(n, inf<T>);
5     vector<int> pre(n, -1);
6
7     dist[s] = 0;
8
9     for (int round = 1; round++;) {
10         bool upd = false;
11         for (int u = 0; u < n; u++) {
12             if (dist[u] == inf<T>) {
13                 continue;
14             }
15             for (auto& [v, w] : adj[u]) {
16                 if (dist[v] > dist[u] + w) {
17                     dist[v] = dist[u] + w;
18                     pre[v] = u;

```



```

19         upd = true;
20         if (round >= n) {
21             return {{}, {}};
22         }
23     }
24 }
25 }
26 if (!upd) {
27     break;
28 }
29 }
30
31 return {dist, pre};
32 }
33

```

2.2.4 SPFA

```

1 template <typename T>
2 pair<vector<T>, vector<int>> spfa(vector<vector<pair<int, T>>& adj, int s) {
3     int n = adj.size();
4     vector<T> dist(n, inf<T>);
5     vector<int> pre(n, -1);
6
7     dist[s] = 0;
8
9     queue<int> q;
10    vector<char> inq(n, false);
11    vector<int> cnt(n, 0);
12    q.push(s);
13    inq[s] = true;
14
15    while (!q.empty()) {
16        int u = q.front();
17        q.pop();
18        inq[u] = false;
19        for (auto& [v, w] : adj[u]) {
20            if (dist[v] > dist[u] + w) {
21                dist[v] = dist[u] + w;
22                pre[v] = u;
23                if (!inq[v]) {
24                    q.push(v);
25                    inq[v] = true;
26                    cnt[v]++;

```

```

27                if (cnt[v] >= n) {
28                    return {{}, {}};
29                }
30            }
31        }
32    }
33 }
34
35 return {dist, pre};
36 }

```

2.2.5 Floyd

```

1 template <typename T>
2 void floyd(vector<vector<T>>& dist) {
3     int n = dist.size();
4     for (int k = 0; k < n; k++) {
5         for (int i = 0; i < n; i++) {
6             for (int j = 0; j < n; j++) {
7                 if (dist[i][k] < inf<T> && dist[k][j] < inf<T>) {
8                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
9                 }
10            }
11        }
12    }
13 }
14
15 template <typename T>
16 vector<vector<T>> floyd(vector<vector<pair<int, T>>& adj) {
17     int n = adj.size();
18     vector dist(n, vector<T>(n, inf<T>));
19     for (int u = 0; u < n; u++) {
20         dist[u][u] = 0;
21         for (auto& [v, w] : adj[u]) {
22             dist[u][v] = min(dist[u][v], w);
23         }
24     }
25     floyd(dist);
26     return dist;
27 }
28

```

2.2.6 路径重建

```

1 vector<int> restorePath(vector<int>& pre, int t) {

```

```

2  vector<int> path = {t};
3  while (pre[path.back()] != -1) {
4      path.push_back(pre[path.back()]);
5  }
6  reverse(path.begin(), path.end());
7  return path;
8  }

```

2.3 拓扑排序

```

1  template <typename T>
2  vector<int> toposort(vector<vector<pair<int, T>>& adj) {
3      int n = adj.size();
4      vector<int> topo, deg(n, 0);
5      for (int u = 0; u < n; u++) {
6          for (auto& [v, w] : adj[u]) {
7              deg[v]++;
8          }
9      }
10     for (int u = 0; u < n; u++) {
11         if (deg[u] == 0) {
12             topo.push_back(u);
13         }
14     }
15     for (int i = 0; i < int(topo.size()); i++) {
16         int u = topo[i];
17         for (auto& [v, w] : adj[u]) {
18             if (--deg[v] == 0) {
19                 topo.push_back(v);
20             }
21         }
22     }
23     return (int(topo.size()) != n ? vector<int>() : topo);
24 }
25
26 template <typename T>
27 vector<int> lexMinToposort(vector<vector<pair<int, T>>& adj) {
28     int n = adj.size();
29     vector<int> topo, deg(n, 0);
30     for (int u = 0; u < n; u++) {
31         for (auto& [v, w] : adj[u]) {
32             deg[v]++;
33         }
34     }

```

```

35     priority_queue<int, vector<int>, greater<int>> pq;
36     for (int u = 0; u < n; u++) {
37         if (deg[u] == 0) {
38             pq.push(u);
39         }
40     }
41     while (!pq.empty()) {
42         int u = pq.top();
43         pq.pop();
44         topo.push_back(u);
45         for (auto& [v, w] : adj[u]) {
46             if (--deg[v] == 0) {
47                 pq.push(v);
48             }
49         }
50     }
51     return (int(topo.size()) != n ? vector<int>() : topo);
52 }

```

2.4 欧拉通路/回路

给定起点 s ，还需要满足所有有边的顶点属于同一个连通分量，且 s 不能是孤立点。

无向图：

- 奇度点的数量一定是偶数，因此不会出现只有 1 个奇度点的情况。
- 有 0 个奇度点时，存在从任意有边顶点 s 出发的欧拉回路。
- 有 2 个奇度点时， s 必须是其中一个，存在从 s 出发的欧拉通路。
- 奇度点超过 2 个时不存在欧拉通路。

有向图：

- 若所有顶点满足 $\text{in} = \text{out}$ ，存在从任意有边顶点 s 出发的欧拉回路。
- 若恰有一个顶点满足 $\text{out} = \text{in} + 1$ ，它必须是 s ；同时恰有一个顶点满足 $\text{in} = \text{out} + 1$ ，存在从 s 出发的欧拉通路。
- 其他情况不存在欧拉通路。

```

1  // return:
2  // 0 - isn't Eulerian path
3  // 1 - is Eulerian path
4  // 2 - is Eulerian circuit
5  template <typename T>
6  int isEulerian(vector<vector<pair<int, T>>& adj, bool directed = true, int s = -1)
7  {
8      int n = adj.size();
9      // in and out are only for directed graph
10     vector<int> in(n, 0), out(n, 0), deg(n, 0);
11     DSU dsu(n);

```

```

11 for (int u = 0; u < n; u++) {
12     for (auto& [v, w] : adj[u]) {
13         dsu.merge(u, v);
14         if (directed) {
15             out[u]++;
16             in[v]++;
17             deg[v]++;
18         }
19         deg[u]++;
20     }
21 }
22
23 int root = -1;
24 for (int u = 0; u < n; u++) {
25     if (deg[u] == 0) {
26         continue;
27     }
28     if (root == -1) {
29         root = u;
30     } else if (!dsu.same(root, u)) {
31         return 0;
32     }
33 }
34
35 if (s != -1 && root != -1 && !dsu.same(s, root)) {
36     return 0;
37 }
38
39 if (directed) {
40     int diff = 0;
41     for (int u = 0; u < n; u++) {
42         diff += abs(out[u] - in[u]);
43     }
44     if (diff == 0) {
45         return 2;
46     } else if (diff == 2) {
47         if (s == -1) {
48             return 1;
49         } else {
50             return (out[s] == in[s] + 1);
51         }
52     } else {
53         return 0;

```

```

54     }
55 } else {
56     int odd = 0;
57     for (int u = 0; u < n; u++) {
58         odd += (deg[u] & 1);
59     }
60     if (odd == 0) {
61         return 2;
62     } else if (odd == 2) {
63         if (s == -1) {
64             return 1;
65         } else {
66             return (deg[s] & 1);
67         }
68     } else {
69         return 0;
70     }
71 }
72 }
73
74 template <typename T>
75 bool isEulerianPath(vector<vector<pair<int, T>>>& adj, bool directed = true, int s = -1) {
76     return isEulerian(adj, directed, s) >= 1;
77 }
78
79 template <typename T>
80 bool isEulerianCircuit(vector<vector<pair<int, T>>>& adj, bool directed = true, int s = -1) {
81     return isEulerian(adj, directed, s) == 2;
82 }
83
84 template <typename T>
85 pair<vector<int>, vector<Edge<T>>> findEulerian(vector<vector<pair<int, T>>>& adj,
86     int s = -1) {
87     // TODO
88 }

```

2.5 最小生成树

2.5.1 Kruskal

```

1 template <typename T>
2 pair<vector<Edge<T>>, T> kruskal(vector<Edge<T>>& edges, int n) {
3     T cost = 0;
4     vector<int> ord(edges.size());
5     iota(ord.begin(), ord.end(), 0);

```

```

6   sort(ord.begin(), ord.end(), [&](int i, int j) {
7       return edges[i].w < edges[j].w;
8   });
9
10  DSU dsu(n);
11  vector<Edge<T>> res;
12
13  for (auto& id : ord) {
14      auto& e = edges[id];
15      if (dsu.merge(e.u, e.v)) {
16          res.push_back(e);
17          cost += e.w;
18      }
19  }
20
21  return {res, cost};
22 }
23

```

2.5.2 Prim

```

1  template <typename T, typename F>
2  vector<Edge<T>> prim(int n, F cost) {
3      vector<Edge<T>> res;
4      vector<T> weight(n, inf<T>);
5      vector<int> pre(n);
6
7      auto add = [&](int u) -> void {
8          for (int v = 0; v < n; v++) {
9              if (pre[v] == -1) {
10                 continue;
11             }
12             T w = cost(u, v);
13             if (weight[v] > w) {
14                 weight[v] = w;
15                 pre[v] = u;
16             }
17         }
18         weight[u] = inf<T>;
19         pre[u] = -1;
20     };
21
22     add(0);
23     for (int i = 0; i < n - 1; i++) {

```

```

24         int u = min_element(weight.begin(), weight.end()) - weight.begin();
25         res.emplace_back(pre[u], u, weight[u]);
26         add(u);
27     }
28
29     return res;
30 }
31
32 template <typename T>
33 vector<Edge<T>> prim(vector<vector<pair<int, T>>& adj) {
34     int n = adj.size();
35     vector<Edge<T>> res;
36     vector<T> weight(n, inf<T>);
37     vector<int> pre(n);
38
39     auto add = [&](int u) -> void {
40         for (auto& [v, w] : adj[u]) {
41             if (pre[v] == -1) {
42                 continue;
43             }
44             if (weight[v] > w) {
45                 weight[v] = w;
46                 pre[v] = u;
47             }
48         }
49         weight[u] = inf<T>;
50         pre[u] = -1;
51     };
52
53     add(0);
54     for (int i = 0; i < n - 1; i++) {
55         int u = min_element(weight.begin(), weight.end()) - weight.begin();
56         res.emplace_back(pre[u], u, weight[u]);
57         add(u);
58     }
59
60     return res;
61 }

```

2.5.3 Borůvka

```

1  template <typename T>
2  pair<vector<Edge<T>>, T> boruvka(vector<Edge<T>>& edges, int n) {
3      T cost = 0;
4      vector<Edge<T>> res;

```

```

5 DSU dsu(n);
6 int cnt = n;
7
8 while (cnt > 1) {
9     vector<int> best(n, -1);
10    int m = edges.size();
11
12    for (int i = 0; i < m; i++) {
13        auto& e = edges[i];
14        int u = dsu.find(e.u);
15        int v = dsu.find(e.v);
16        if (u == v) {
17            continue;
18        }
19        if (best[u] == -1 || e.w < edges[best[u]].w) {
20            best[u] = i;
21        }
22        if (best[v] == -1 || e.w < edges[best[v]].w) {
23            best[v] = i;
24        }
25    }
26
27    bool upd = false;
28    for (int u = 0; u < n; u++) {
29        if (best[u] == -1) {
30            continue;
31        }
32        auto& e = edges[best[u]];
33        if (dsu.merge(e.u, e.v)) {
34            res.push_back(e);
35            cost += e.w;
36            cnt--;
37            upd = true;
38        }
39    }
40    if (!upd) {
41        break;
42    }
43 }
44
45 return {res, cost};
46 }

```

2.5.4 Kruskal 重构树

```

1 template <typename T>
2 struct KruskalTree {
3     int n, root;
4     vector<i64> val;
5     vector<Edge<T>> edges;
6     vector<vector<int>> adj;
7
8     KruskalTree(int n_) {
9         init(n_);
10    }
11
12    void init(int n_) {
13        n = n_;
14        root = n - 1;
15        val.assign(2 * n - 1, 0);
16        adj.assign(2 * n - 1, vector<int>{});
17        edges.clear();
18    }
19
20    void addEdge(int u, int v, i64 w) {
21        edges.emplace_back(u, v, w);
22    }
23
24    void build() {
25        DSU dsu(2 * n - 1);
26        vector<int> ord(2 * n - 1);
27        iota(ord.begin(), ord.end(), 0);
28        sort(ord.begin(), ord.end(), [&](int x, int y) {
29            return edges[x].w < edges[y].w;
30        });
31
32        for (auto& id : ord) {
33            auto& e = edges[id];
34            int u = e.u, v = e.v;
35            i64 w = e.w;
36
37            if (!dsu.same(u, v)) {
38                u = dsu.find(u);
39                v = dsu.find(v);
40                val[++root] = w;
41
42                adj[root].push_back(u);
43                adj[root].push_back(v);

```

```

44     adj[u].push_back(root);
45     adj[v].push_back(root);
46
47     dsu.f[u] = root;
48     dsu.f[v] = root;
49     dsu.siz[root] = dsu.siz[u] + dsu.siz[v];
50
51     if (root == 2 * n - 2) {
52         return;
53     }
54 }
55 }
56 }
57 };

```

2.6 连通性

2.6.1 强连通分量

```

1 struct SCC {
2     int n;
3     vector<vector<int>>> adj;
4     vector<int> stk;
5     vector<int> dfn, low, bel;
6     int cur, cnt;
7
8     SCC() {}
9     SCC(int n_) {
10         init(n_);
11     }
12
13     void init(int n_) {
14         n = n_;
15         adj.assign(n, {});
16         dfn.assign(n, -1);
17         low.resize(n);
18         bel.assign(n, -1);
19         stk.clear();
20         cur = cnt = 0;
21     }
22
23     void addEdge(int u, int v) {
24         adj[u].push_back(v);
25     }
26

```

```

27 void dfs(int u) {
28     dfn[u] = low[u] = cur++;
29     stk.push_back(u);
30
31     for (auto& v : adj[u]) {
32         if (dfn[v] == -1) {
33             dfs(v);
34             low[u] = min(low[u], low[v]);
35         } else if (bel[v] == -1) {
36             low[u] = min(low[u], dfn[v]);
37         }
38     }
39
40     if (dfn[u] == low[u]) {
41         int y;
42         do {
43             y = stk.back();
44             bel[y] = cnt;
45             stk.pop_back();
46         } while (y != u);
47         cnt++;
48     }
49 }
50
51 vector<int> work() {
52     for (int i = 0; i < n; i++) {
53         if (dfn[i] == -1) {
54             dfs(i);
55         }
56     }
57     for (auto& x : bel) { // 把逆拓扑序转成拓扑序
58         x = cnt - 1 - x;
59     }
60     return bel;
61 }
62 };

```

2.6.2 边双连通分量

```

1 struct EBCC {
2     int n;
3     vector<vector<pair<int, int>>>> adj;
4     vector<int> stk;
5     vector<int> dfn, low, bel;

```

```

6  int cur, cnt, ecnt;
7
8  EBCC() {}
9  EBCC(int n_) {
10     init(n_);
11 }
12
13 void init(int n_) {
14     n = n_;
15     adj.assign(n, {});
16     dfn.assign(n, -1);
17     low.resize(n);
18     bel.assign(n, -1);
19     stk.clear();
20     cur = cnt = ecnt = 0;
21 }
22
23 void addEdge(int u, int v) {
24     adj[u].emplace_back(v, ecnt);
25     adj[v].emplace_back(u, ecnt);
26     ecnt++;
27 }
28
29 void dfs(int u, int p) {
30     dfn[u] = low[u] = cur++;
31     stk.push_back(u);
32
33     for (auto& [v, eid] : adj[u]) {
34         if (eid == p) {
35             continue;
36         }
37         if (dfn[v] == -1) {
38             dfs(v, eid);
39             low[u] = min(low[u], low[v]);
40         } else {
41             low[u] = min(low[u], dfn[v]);
42         }
43     }
44
45     if (dfn[u] == low[u]) {
46         int y;
47         do {
48             y = stk.back();

```

```

49         bel[y] = cnt;
50         stk.pop_back();
51     } while (y != u);
52     cnt++;
53 }
54 }
55
56 vector<int> work() {
57     for (int i = 0; i < n; i++) {
58         if (dfn[i] == -1) {
59             dfs(i, -1);
60         }
61     }
62     return bel;
63 }
64 };

```

2.6.3 点双连通分量

```

1 struct VBCC {
2     int n;
3     vector<pair<int, int>> edges;
4     vector<vector<pair<int, int>>> adj;
5     vector<int> stk;
6     vector<int> dfn, low, cut;
7     vector<vector<int>> comp;
8     int cur;
9
10    VBCC() {}
11    VBCC(int n_) {
12        init(n_);
13    }
14
15    void init(int n_) {
16        n = n_;
17        adj.assign(n, {});
18        edges.clear();
19        dfn.assign(n, -1);
20        low.resize(n);
21        stk.clear();
22        comp.clear();
23        cut.assign(n, false);
24        cur = 0;
25    }

```

```

26
27 void addEdge(int u, int v) {
28     adj[u].emplace_back(v, edges.size());
29     adj[v].emplace_back(u, edges.size());
30     edges.emplace_back(u, v);
31 }
32
33 void dfs(int u, int p) {
34     dfn[u] = low[u] = cur++;
35     int child = 0;
36     for (auto& [v, eid] : adj[u]) {
37         if (eid == p) {
38             continue;
39         }
40         if (dfn[v] == -1) {
41             child++;
42             stk.push_back(eid);
43             dfs(v, eid);
44             low[u] = min(low[u], low[v]);
45             if (low[v] >= dfn[u]) {
46                 if (p != -1) {
47                     cut[u] = true;
48                 }
49                 vector<int> b;
50                 while (true) {
51                     int e = stk.back();
52                     stk.pop_back();
53                     b.push_back(e);
54                     if (e == eid) {
55                         break;
56                     }
57                 }
58                 comp.push_back(b);
59             }
60         } else if (dfn[v] < dfn[u]) {
61             stk.push_back(eid);
62             low[u] = min(low[u], dfn[v]);
63         }
64     }
65     if (p == -1 && child >= 2) {
66         cut[u] = true;
67     }
68 }

```

```

69
70 void work() {
71     for (int i = 0; i < n; i++) {
72         if (dfn[i] == -1) {
73             dfs(i, -1);
74         }
75     }
76 }
77 };

```

2.6.4 圆方树

```

1 struct BlockCutTree {
2     int n;
3     vector<pair<int, int>> edges;
4     vector<vector<pair<int, int>>> adj;
5     vector<int> stk;
6     vector<int> dfn, low, cut;
7     vector<vector<int>> comp;
8     vector<vector<int>> tree;
9     int cur;
10    int tot;
11
12    BlockCutTree() {}
13    BlockCutTree(int n_) {
14        init(n_);
15    }
16
17    void init(int n_) {
18        n = n_;
19        adj.assign(n, {});
20        edges.clear();
21        dfn.assign(n, -1);
22        low.resize(n);
23        stk.clear();
24        comp.clear();
25        cut.assign(n, false);
26        tree.assign(n, {});
27        tot = n;
28        cur = 0;
29    }
30
31    void addEdge(int u, int v) {
32        adj[u].emplace_back(v, edges.size());

```



```

33     adj[v].emplace_back(u, edges.size());
34     edges.emplace_back(u, v);
35 }
36
37 void addBlock(vector<int>& b) {
38     int x = tot++;
39     tree.emplace_back();
40     vector<int> pts;
41     for (auto& e : b) {
42         auto [u, v] = edges[e];
43         pts.push_back(u);
44         pts.push_back(v);
45     }
46     sort(pts.begin(), pts.end());
47     pts.erase(unique(pts.begin(), pts.end()), pts.end());
48     for (int u : pts) {
49         tree[u].push_back(x);
50         tree[x].push_back(u);
51     }
52 }
53
54 void dfs(int u, int p) {
55     dfn[u] = low[u] = cur++;
56     int child = 0;
57     for (auto& [v, eid] : adj[u]) {
58         if (eid == p) {
59             continue;
60         }
61         if (dfn[v] == -1) {
62             child++;
63             stk.push_back(eid);
64             dfs(v, eid);
65             low[u] = min(low[u], low[v]);
66             if (low[v] >= dfn[u]) {
67                 if (p != -1) {
68                     cut[u] = true;
69                 }
70                 vector<int> b;
71                 while (true) {
72                     int e = stk.back();
73                     stk.pop_back();
74                     b.push_back(e);
75                     if (e == eid) {

```

```

76                         break;
77                     }
78                 }
79                 comp.push_back(b);
80                 addBlock(b);
81             }
82             } else if (dfn[v] < dfn[u]) {
83                 stk.push_back(eid);
84                 low[u] = min(low[u], dfn[v]);
85             }
86         }
87         if (p == -1 && child >= 2) {
88             cut[u] = true;
89         }
90     }
91
92 void build() {
93     for (int i = 0; i < n; i++) {
94         if (dfn[i] == -1) {
95             dfs(i, -1);
96         }
97     }
98 }
99 };

```

2.7 树链剖分

```

1 struct HLD {
2     int n;
3     vector<int> siz, top, dep, fa, in, out, seq;
4     vector<vector<int>> adj;
5     int cur;
6
7     HLD() {}
8     HLD(int n_) {
9         n = n_;
10        siz.resize(n);
11        top.resize(n);
12        dep.resize(n);
13        fa.resize(n);
14        in.resize(n);
15        out.resize(n);
16        seq.resize(n);
17        cur = 0;

```

```

18     adj.assign(n, {});
19 }
20
21 void addEdge(int u, int v) {
22     adj[u].push_back(v);
23     adj[v].push_back(u);
24 }
25
26 void work(int root = 0) {
27     top[root] = root;
28     dep[root] = 0;
29     fa[root] = -1;
30     dfs1(root);
31     dfs2(root);
32 }
33
34 void dfs1(int u) {
35     if (fa[u] != -1) {
36         adj[u].erase(find(adj[u].begin(), adj[u].end(), fa[u]));
37     }
38
39     siz[u] = 1;
40     for (auto& v : adj[u]) {
41         fa[v] = u;
42         dep[v] = dep[u] + 1;
43         dfs1(v);
44         siz[u] += siz[v];
45         if (siz[v] > siz[adj[u][0]]) {
46             swap(v, adj[u][0]);
47         }
48     }
49 }
50
51 void dfs2(int u) {
52     in[u] = cur++;
53     seq[in[u]] = u;
54     for (auto& v : adj[u]) {
55         top[v] = (v == adj[u][0] ? top[u] : v);
56         dfs2(v);
57     }
58     out[u] = cur;
59 }
60

```

```

61 int lca(int u, int v) {
62     while (top[u] != top[v]) {
63         if (dep[top[u]] > dep[top[v]]) {
64             u = fa[top[u]];
65         } else {
66             v = fa[top[v]];
67         }
68     }
69     return dep[u] < dep[v] ? u : v;
70 }
71
72 int dist(int u, int v) {
73     return dep[u] + dep[v] - 2 * dep[lca(u, v)];
74 }
75
76 int jump(int u, int k) {
77     if (dep[u] < k) {
78         return -1;
79     }
80     int d = dep[u] - k;
81     while (dep[top[u]] > d) {
82         u = fa[top[u]];
83     }
84     return seq[in[u] - dep[u] + d];
85 }
86
87 bool isAncestor(int u, int v) {
88     return in[u] <= in[v] && in[v] < out[u];
89 }
90
91 int rootedParent(int u, int v) {
92     swap(u, v);
93     if (u == v) {
94         return u;
95     }
96     if (!isAncestor(u, v)) {
97         return fa[u];
98     }
99     auto it = upper_bound(adj[u].begin(), adj[u].end(), v, [&](int x, int y)
100 { return in[x] < in[y]; }) - 1;
101     return *it;
102 }
103
104 int rootedSize(int u, int v) {

```

```

104     if (u == v) {
105         return n;
106     }
107     if (!isAncestor(v, u)) {
108         return siz[v];
109     }
110     return n - siz[rootedParent(u, v)];
111 }
112
113 int rootedLca(int a, int b, int c) {
114     return lca(a, b) ^ lca(b, c) ^ lca(c, a);
115 }
116 };
117

```

2.8 LCA

```

1 struct LCA {
2     int logn, cur;
3     vector<int> dep, dfn;
4     vector<vector<int>>> fa;
5
6     LCA(vector<vector<int>>& adj, int root = 0) {
7         init(adj, root);
8     }
9
10    void init(vector<vector<int>>& adj, int root) {
11        int n = adj.size();
12        cur = 0;
13        logn = __lg(n);
14        fa.assign(n, vector<int>(logn + 1, -1));
15        dep.assign(n, 0);
16        dfn.assign(n, -1);
17
18        auto dfs = [&](auto&& self, int u, int p) -> void {
19            fa[u][0] = p;
20            dfn[u] = cur++;
21            for (int i = 1; i <= logn; i++) {
22                fa[u][i] = fa[fa[u][i - 1]][i - 1];
23            }
24            for (auto& v : adj[u]) {
25                if (v == p) {
26                    continue;
27                }
28            }
29        };
30        dfs(root, root);
31    }
32
33    int get(int u, int v) {
34        if (dep[u] < dep[v]) {
35            swap(u, v);
36        }
37        for (int i = logn; i >= 0; i--) {
38            if (dep[fa[u][i]] >= dep[v]) {
39                u = fa[u][i];
40            }
41        }
42        if (u == v) {
43            return u;
44        }
45        for (int i = logn; i >= 0; i--) {
46            if (fa[u][i] != fa[v][i]) {
47                u = fa[u][i];
48                v = fa[v][i];
49            }
50        }
51        return fa[u][0];
52    }
53
54    int dist(int u, int v) {
55        return dep[u] + dep[v] - 2 * dep[get(u, v)];
56    }
57 };
58

```

```

28         dep[v] = dep[u] + 1;
29         self(self, v, u);
30     }
31 };
32
33 dfs(dfs, root, root);
34
35 for (int i = n - 1; i >= 0; i--) {
36     if (fa[i][0] == -1) {
37         dfs(dfs, i, i);
38     }
39 }
40
41
42 int get(int u, int v) {
43     if (dep[u] < dep[v]) {
44         swap(u, v);
45     }
46     for (int i = logn; i >= 0; i--) {
47         if (dep[fa[u][i]] >= dep[v]) {
48             u = fa[u][i];
49         }
50     }
51     if (u == v) {
52         return u;
53     }
54     for (int i = logn; i >= 0; i--) {
55         if (fa[u][i] != fa[v][i]) {
56             u = fa[u][i];
57             v = fa[v][i];
58         }
59     }
60     return fa[u][0];
61 }
62
63 int dist(int u, int v) {
64     return dep[u] + dep[v] - 2 * dep[get(u, v)];
65     return res;
66 }
67 };
68

```

2.9 虚树

```

1 template <typename T>
2 struct VirtualTree {
3     using Graph = vector<vector<pair<int, T>>>;
4
5     LCA& lca;
6     vector<int> nodes;
7     Graph adj;
8
9     VirtualTree(LCA& lca) : lca(lca) {}
10
11     pair<vector<int>&, Graph&> build(const vector<int>& h) {
12         nodes = h;
13         auto cmp = [&](int u, int v) { return lca.dfn[u] < lca.dfn[v]; };
14         sort(nodes.begin(), nodes.end(), cmp);
15         int m = nodes.size();
16         for (int i = 1; i < m; i++) {
17             nodes.push_back(lca.get(nodes[i - 1], nodes[i]));
18         }
19         sort(nodes.begin(), nodes.end(), cmp);
20         nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
21         int n = nodes.size();
22         adj.resize(n);
23         for (auto& e : adj) {
24             e.clear();
25         }
26         for (int i = 0; i < n - 1; i++) {
27             int p = lca.get(nodes[i], nodes[i + 1]);
28             int u = lower_bound(nodes.begin(), nodes.end(), p, cmp) -
nodes.begin();
29             int v = i + 1;
30             T w = lca.dep[nodes[v]] - lca.dep[p];
31             adj[u].emplace_back(v, w);
32             adj[v].emplace_back(u, w);
33         }
34
35         return {nodes, adj};
36     }
37 };

```

2.10 2-SAT

addClause(u, f, v, g) 加入一个析取子句:

$$(x_u = f) \vee (x_v = g).$$

这里 x_i 是第 i 个布尔变量, 变量编号从 0 开始。f 和 g 决定文字的正负: true 表示正文字 x_i , false 表示负文字 $\neg x_i$ 。

例如: addClause(3, true, 5, false) 表示

$$x_3 \vee \neg x_5.$$

一个子句 $(A \vee B)$ 等价于两条蕴含边:

$$(\neg A) \rightarrow B, (\neg B) \rightarrow A.$$

所以 addClause(u, f, v, g) 实际加入:

$$(x_u = \neg f) \rightarrow (x_v = g), (x_v = \neg g) \rightarrow (x_u = f).$$

图中的编号约定是: $2i$ 表示变量 x_i 取假值, $2i+1$ 表示变量 x_i 取真值。因此第一条边的起点是第一个文字的否定, 终点是第二个文字本身; 另一条边同理。

```

1 struct TwoSat {
2     int n;
3     SCC scc;
4     vector<int> ans;
5
6     TwoSat(int n_) : n(n_), scc(2 * n), ans(n) {}
7
8     // add clause: (x_u = f) ∨ (x_v = g)
9     void addClause(int u, bool f, int v, bool g) {
10         scc.addEdge(2 * u + !f, 2 * v + g);
11         scc.addEdge(2 * v + !g, 2 * u + f);
12     }
13
14     // (x_u = f)
15     void set(int u, bool f) {
16         addClause(u, f, u, f);
17     }
18
19     // (x_u = f) -> (x_v = g)
20     void implies(int u, int f, int v, int g) {
21         addClause(u, !f, v, g);
22     }
23
24     pair<bool, vector<int>> satisfiable() {
25         auto id = scc.work();
26         for (int i = 0; i < n; i++) {
27             if (id[2 * i] == id[2 * i + 1]) {
28                 return {false, {}};
29             }
30             ans[i] = id[2 * i] < id[2 * i + 1];
31         }
32         return {true, ans};
33     }

```

```
34 };
```

3 网络流

3.1 最大流最小割

```
1 template <typename T>
2 struct MaxFlow {
3     struct Edge_ {
4         int to;
5         T cap;
6         Edge_(int to_, T cap_) : to(to_), cap(cap_) {}
7     };
8
9     int n;
10    vector<Edge_> e;
11    vector<vector<int>> adj;
12    vector<int> cur, level;
13
14    MaxFlow() {}
15    MaxFlow(int n_) {
16        init(n_);
17    }
18
19    void init(int n_) {
20        n = n_;
21        e.clear();
22        adj.assign(n_, {});
23        cur.resize(n_);
24        level.resize(n_);
25    }
26
27    bool bfs(int s, int t) {
28        level.assign(n, -1);
29        queue<int> q;
30        level[s] = 0;
31        q.push(s);
32        while (!q.empty()) {
33            const int u = q.front();
34            q.pop();
35            for (auto& i : adj[u]) {
36                auto [v, c] = e[i];
37                if (c > 0 && level[v] == -1) {
38                    level[v] = level[u] + 1;
```

```
39                if (v == t) {
40                    return true;
41                }
42                q.push(v);
43            }
44        }
45        return false;
46    }
47
48    T dfs(int u, int t, T f) {
49        if (u == t) {
50            return f;
51        }
52        auto r = f;
53        for (int& i = cur[u]; i < int(adj[u].size()); i++) {
54            const int& j = adj[u][i];
55            auto [v, c] = e[j];
56            if (c > 0 && level[v] == level[u] + 1) {
57                auto a = dfs(v, t, min(r, c));
58                e[j].cap -= a;
59                e[j ^ 1].cap += a;
60                r -= a;
61                if (r == 0) {
62                    return f;
63                }
64            }
65        }
66        return f - r;
67    }
68
69    void addEdge(int u, int v, T c) {
70        adj[u].push_back(e.size());
71        e.emplace_back(v, c);
72        adj[v].push_back(e.size());
73        e.emplace_back(u, 0);
74    }
75
76    T flow(int s, int t) {
77        T ans = 0;
78        while (bfs(s, t)) {
79            cur.assign(n, 0);
80            ans += dfs(s, t, numeric_limits<T>::max());
81        }
```

```

82     }
83     return ans;
84 }
85
86 vector<bool> minCut() {
87     vector<bool> c(n);
88     for (int i = 0; i < n; i++) {
89         c[i] = (level[i] != -1);
90     }
91     return c;
92 }
93
94 struct Edge {
95     int from;
96     int to;
97     T cap;
98     T flow;
99 };
100
101 vector<Edge> edges() {
102     vector<Edge> a;
103     for (int i = 0; i < e.size(); i += 2) {
104         Edge x;
105         x.from = e[i + 1].to;
106         x.to = e[i].to;
107         x.cap = e[i].cap + e[i + 1].cap;
108         x.flow = e[i + 1].cap;
109         a.push_back(x);
110     }
111     return a;
112 }
113 };

```

3.2 最小费用最大流

最小费用流的每次增广都要在残量网络上找最短路，而残量网络里有反向边、费用是负的，Dijkstra 直接用不了。势能数组 h 就是用来把边权「平移」成非负的。

做法是把每条边的费用换成约化费用

$$c'(u, v) = c(u, v) + h(u) - h(v),$$

并且每轮求完最短路后令 $h(v) \leftarrow h(v) + \text{dist}'(v)$ ，其中 dist' 是用约化费用算出的最短路。这样之后的约化费用依然非负——由三角不等式 $\text{dist}'(v) \leq \text{dist}'(u) + c'(u, v)$ 整理一下，正好就是新的 $c'(u, v) \geq 0$ 。

代价是路径长度被平移了常量 $h(s) - h(t)$ ，但它不影响谁是最短路。代码里每轮累加的是 $h(t)$ ：因为 $h(s)$ 恒为 0，更新后的 $h(t)$ 恰好等于原网络中 $s \rightarrow t$ 的真实最短费用。

初始势能默认全 0，这就要求**初始边权全部非负**；如果原图本来就有负费用边，得根据图的特性自行构建 h 或者直接先跑一遍 SPFA 得到初始 h ，再从参数传进来。

```

1 template<typename T>
2 struct MinCostFlow {
3     struct Edge_ {
4         int to;
5         T cap;
6         T cost;
7         Edge_(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
8     };
9
10    int n;
11    vector<Edge_> e;
12    vector<vector<int>>> adj;
13    vector<T> h, dist;
14    vector<int> pre;
15
16    bool dijkstra(int s, int t) {
17        dist.assign(n, numeric_limits<T>::max());
18        pre.assign(n, -1);
19        priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>>
20    que;
21
22    dist[s] = 0;
23    que.emplace(0, s);
24    while (!que.empty()) {
25        auto [d, u] = que.top();
26        que.pop();
27        if (dist[u] != d) {
28            continue;
29        }
30        for (auto& i : adj[u]) {
31            int v = e[i].to;
32            if (e[i].cap > 0 && dist[v] > d + h[u] - h[v] + e[i].cost) {
33                dist[v] = d + h[u] - h[v] + e[i].cost;
34                pre[v] = i;
35                que.emplace(dist[v], v);
36            }
37        }
38    }
39
40    return dist[t] != numeric_limits<T>::max();
41 }
42
43 MinCostFlow() {}
44 MinCostFlow(int n_) {

```

```

42     init(n_);
43 }
44
45 void init(int n_) {
46     n = n_;
47     e.clear();
48     adj.assign(n, {});
49 }
50
51 void addEdge(int u, int v, T cap, T cost) {
52     adj[u].push_back(e.size());
53     e.emplace_back(v, cap, cost);
54     adj[v].push_back(e.size());
55     e.emplace_back(u, 0, -cost);
56 }
57
58 pair<T, T> flow(int s, int t, T need = numeric_limits<T>::max(), vector<T>
potential = {}) {
59     T flow = 0;
60     T cost = 0;
61     if (potential.empty()) {
62         potential.assign(n, 0);
63     }
64     h = potential;
65     while (flow < need && dijkstra(s, t)) {
66         for (int i = 0; i < n; ++i) {
67             h[i] += dist[i];
68         }
69         T aug = numeric_limits<T>::max();
70         for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
71             aug = min(aug, e[pre[i]].cap);
72         }
73         for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
74             e[pre[i]].cap -= aug;
75             e[pre[i] ^ 1].cap += aug;
76         }
77         flow += aug;
78         cost += aug * h[t];
79     }
80     return {flow, cost};
81 }
82
83 struct Edge {

```

```

84     int from;
85     int to;
86     T cap;
87     T cost;
88     T flow;
89 };
90
91 vector<Edge> edges() {
92     vector<Edge> a;
93     for (int i = 0; i < e.size(); i += 2) {
94         Edge x;
95         x.from = e[i + 1].to;
96         x.to = e[i].to;
97         x.cap = e[i].cap + e[i + 1].cap;
98         x.cost = e[i].cost;
99         x.flow = e[i + 1].cap;
100         a.push_back(x);
101     }
102     return a;
103 }
104 };

```

4 数论

4.1 扩展欧几里得算法

```

1 i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
2     if (b == 0) {
3         x = 1, y = 0;
4         return a;
5     }
6     i64 d = exgcd(b, a % b, y, x);
7     y -= a / b * x;
8     return d;
9 }
10
11 i64 inv(i64 a, i64 m) {
12     i64 x, y;
13     i64 d = exgcd(a, m, x, y);
14     if (d != 1) {
15         return -1;
16     }
17     return (x % m + m) % m;
18 }

```

```

19
20 bool solveLinear(i64 a, i64 b, i64 c, i64& x, i64& y) {
21     i64 d = exgcd(a, b, x, y);
22     if (c % d != 0) {
23         return false;
24     }
25     i64 k = c / d;
26     x *= k;
27     y *= k;
28     return true;
29 }
30
31 pair<i64, i64> solveMod(i64 a, i64 b, i64 m) {
32     assert(m > 0);
33     b *= -1;
34     i64 x, y;
35     i64 g = exgcd(a, m, x, y);
36     if (g < 0) {
37         g *= -1;
38         x *= -1;
39         y *= -1;
40     }
41     if (b % g != 0) {
42         return {-1, -1};
43     }
44     x = x * (b / g) % (m / g);
45     if (x < 0) {
46         x += m / g;
47     }
48     return {x, m / g};
49 }
50

```

4.2 线性筛

```

1 vector<int> minp, primes;
2
3 void sieve(int n) {
4     minp.assign(n + 1, 0);
5     primes.clear();
6
7     for (int i = 2; i <= n; i++) {
8         if (minp[i] == 0) {
9             minp[i] = i;

```

```

10         primes.push_back(i);
11     }
12
13     for (auto p : primes) {
14         if (i * p > n) {
15             break;
16         }
17         minp[i * p] = p;
18         if (p == minp[i]) {
19             break;
20         }
21     }
22 }
23

```

4.3 莫比乌斯反演

莫比乌斯反演用来在「按约数求和」和「按倍数求和」之间互转。

$\mu(n)$ 的定义: $n = 1$ 时为 1; n 是 k 个互异素数之积时为 $(-1)^k$; 含平方因子时为 0。一切从这条恒等式出发:

$$\sum_{d|n} \mu(d) = [n = 1].$$

于是有两种方向的反演:

$$f(n) = \sum_{d|n} g(d) \implies g(n) = \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right),$$

$$f(n) = \sum_{n|d} g(d) \implies g(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) f(d).$$

实际做题更常用的是「换条件」的用法: 把 $[\gcd(i, j) = 1]$ 换成 $\sum_{d|\gcd(i, j)} \mu(d)$, 再交换求和顺序, 条件就从 gcd 挪到了 d :

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor.$$

右边只要 μ 的前缀和就能整除分块, μ 用下面的 `sieve()` 线性筛出来。

```

1 vector<int> minp, primes, mu;
2
3 void sieve(int n) {
4     minp.assign(n + 1, 0);
5     mu.assign(n + 1, 0);
6     primes.clear();
7     mu[1] = 1;
8
9     for (int i = 2; i <= n; i++) {
10         if (minp[i] == 0) {
11             minp[i] = i;

```



```

12     mu[i] = -1;
13     primes.push_back(i);
14 }
15
16 for (auto p : primes) {
17     if (i * p > n) {
18         break;
19     }
20     minp[i * p] = p;
21     if (p == minp[i]) {
22         mu[i * p] = 0;
23         break;
24     } else {
25         mu[i * p] = -mu[i];
26     }
27 }
28 }
29 }
30

```

4.4 欧拉函数

$$\varphi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right).$$

两条常用性质：它是积性的，即 $\gcd(a, b) = 1$ 时 $\varphi(ab) = \varphi(a)\varphi(b)$ ；并且 $\sum_{d|n} \varphi(d) = n$ ，也即 $\varphi(n) = \sum_{d|n} \mu(d) \frac{n}{d}$ 。

欧拉定理： $\gcd(a, m) = 1$ 时 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。竞赛里更常用的是扩展欧拉定理，它给指数降幂—— $b \geq \varphi(m)$ 时对任意 a 都有

$$a^b \equiv a^{b \bmod \varphi(m) + \varphi(m)} \pmod{m}.$$

线性筛求 φ 的递推：素数的 $\varphi(p) = p - 1$ ；对 $i \cdot p$ ，若 $p \nmid i$ 则 $\varphi(ip) = p\varphi(i)$ ，否则 $\varphi(ip) = (p - 1)\varphi(i)$ 。见下面的 `sieve()`；只求单个数用 `Phi()`，复杂度 $O(\sqrt{n})$ 。

```

1 vector<int> minp, primes, phi;
2
3 // 线性筛（用于求解全部数的欧拉函数）
4 void sieve(int n) {
5     minp.assign(n + 1, 0);
6     phi.assign(n + 1, 0);
7     primes.clear();
8     phi[1] = 1;
9
10    for (int i = 2; i <= n; i++) {
11        if (minp[i] == 0) {
12            minp[i] = i;

```

```

13        primes.push_back(i);
14        phi[i] = i - 1;
15    }
16
17    for (auto p : primes) {
18        if (i * p > n) {
19            break;
20        }
21        minp[i * p] = p;
22        if (p == minp[i]) {
23            phi[i * p] = phi[i] * p;
24            break;
25        } else {
26            phi[i * p] = phi[i] * (p - 1);
27        }
28    }
29 }
30 }
31
32 // 求解单个数的欧拉函数
33 int Phi(int n) {
34     int res = n;
35     for (int i = 2; i * i <= n; i++) {
36         if (n % i == 0) {
37             while (n % i == 0) {
38                 n /= i;
39             }
40             res = res / i * (i - 1);
41         }
42     }
43     if (n > 1) {
44         res = res / n * (n - 1);
45     }
46     return res;
47 }
48

```

4.5 Miller-Rabin

```

1 bool isPrime(i64 n) {
2     if (n < 2) {
3         return false;
4     }
5     constexpr array<int, 10> smallPrimes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};

```

```

6   for (auto& p : smallPrimes) {
7       if (n % p == 0) {
8           return n == p;
9       }
10  }
11  if (n < 31 * 31) {
12      return true;
13  }
14  constexpr array<int, 7> bases = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};
15  int s = __builtin_ctzll(n - 1);
16  i64 d = (n - 1) >> s;
17  for (auto& a : bases) {
18      if (a % n == 0) {
19          continue;
20      }
21      i64 cur = power(a, d, n);
22      if (cur == 1) {
23          continue;
24      }
25      bool witness = true;
26      for (int r = 0; r < s; r++) {
27          if (cur == n - 1) {
28              witness = false;
29              break;
30          }
31          cur = mul(cur, cur, n);
32      }
33      if (witness) {
34          return false;
35      }
36  }
37  return true;
38 }

```

4.6 Pollard-Rho

```

1  static mt19937_64
   rng(static_cast<u32>(chrono::steady_clock::now().time_since_epoch().count()));
2
3  i64 rho(i64 n, i64 c) {
4      auto f = [&](i64 x) {
5          return static_cast<i64>((static_cast<i128>(x) * x + c) % n);
6      };

```

```

7      i64 x = 1, y = 2, z = 1, q = 1;
8      i64 g = 1;
9      constexpr i64 m = 127;
10     for (i64 r = 1; g == 1; r <= 1) {
11         x = y;
12         for (i64 i = 0; i < r; i++) {
13             y = f(y);
14         }
15         for (i64 k = 0; k < r && g == 1; k += m) {
16             z = y;
17             for (i64 i = 0; i < min(m, r - k); i++) {
18                 y = f(y);
19                 q = mul(q, abs(x - y), n);
20             }
21             g = gcd(q, n);
22         }
23     }
24     if (g == n) {
25         do {
26             z = f(z);
27             g = gcd(abs(x - z), n);
28         } while (g == 1);
29     }
30     return g;
31 }
32
33 i64 primeFactor(i64 n) {
34     assert(n >= 2);
35     if (n % 2 == 0) {
36         return 2;
37     }
38     while (!isPrime(n)) {
39         uniform_int_distribution<i64> pick(1, n - 1);
40         n = rho(n, pick(rng));
41     }
42     return n;
43 }
44
45 vector<pair<i64, int>> factorize(i64 n) {
46     vector<pair<i64, int>> pf;
47     for (int p = 2; p < 100; p++) {
48         if (p * p > n) {
49             break;

```

```

50     }
51     if (n % p == 0) {
52         int e = 0;
53         do {
54             n /= p;
55             e++;
56         } while (n % p == 0);
57         pf.emplace_back(p, e);
58     }
59 }
60 while (n > 1) {
61     i64 p = primeFactor(n);
62     int e = 0;
63     do {
64         n /= p;
65         e++;
66     } while (n % p == 0);
67     pf.emplace_back(p, e);
68 }
69 sort(pf.begin(), pf.end());
70 return pf;
71 }

```

4.7 约数构造

```

1 vector<i64> buildDivisors(const vector<pair<i64, int>>& factors) {
2     vector<i64> divisors = {1};
3     for (auto& [p, e] : factors) {
4         int sz = divisors.size();
5         for (int i = 0; i < sz; i++) {
6             i64 x = divisors[i];
7             for (int j = 0; j < e; j++) {
8                 x *= p;
9                 divisors.push_back(x);
10            }
11        }
12    }
13    return divisors;
14 }

```

5 组合数学

5.1 组合数

```

1 template <typename T>

```

```

2 struct Combination {
3     int n;
4     vector<T> fact_, invfact_, inv_;
5
6     Combination() : n(0), fact_{1}, invfact_{1}, inv_{0} {}
7     Combination(int n_) : Combination() {
8         init(n_);
9     }
10
11     void init(int n_) {
12         if (n_ <= n) {
13             return;
14         }
15         fact_.resize(n_ + 1);
16         invfact_.resize(n_ + 1);
17         inv_.resize(n_ + 1);
18
19         for (int i = n + 1; i <= n_; i++) {
20             fact_[i] = fact_[i - 1] * i;
21         }
22         invfact_[n_] = fact_[n_].inv();
23         for (int i = n_; i > n; i--) {
24             invfact_[i - 1] = invfact_[i] * i;
25             inv_[i] = invfact_[i] * fact_[i - 1];
26         }
27         n = n_;
28     }
29
30     T fact(int k) {
31         if (k > n) {
32             init(2 * k);
33         }
34         return fact_[k];
35     }
36     T invfact(int k) {
37         if (k > n) {
38             init(2 * k);
39         }
40         return invfact_[k];
41     }
42     T inv(int k) {
43         if (k > n) {
44             init(2 * k);

```

```

45     }
46     return inv_[k];
47 }
48 T binom(int n, int m) {
49     if (n < m || m < 0) {
50         return 0;
51     }
52     return fact(n) * invfact(m) * invfact(n - m);
53 }
54 T perm(int n, int m) {
55     if (n < m || m < 0) {
56         return 0;
57     }
58     return fact(n) * invfact(n - m);
59 }
60 T multicom(b(int n, int m) {
61     if (n < 0 || m < 0) {
62         return 0;
63     }
64     if (n == 0) {
65         return (m == 0);
66     }
67     return binom(n + m - 1, m);
68 }
69 T Catalan(int n) {
70     if (n < 0) {
71         return 0;
72     }
73     return binom(2 * n, n) * inv(n + 1);
74 }
75 };

```

5.2 康托展开

```

1 vector<int> toLehmer(vector<int>& p) {
2     int n = p.size();
3     Fenwick<AddMono<int>> fen(n);
4     vector<int> c(n);
5     for (int i = n - 1; i >= 0; i--) {
6         c[i] = fen.sum(p[i]);
7         fen.add(p[i], 1);
8     }
9     return c;

```

```

10 }
11
12 vector<int> fromLehmer(vector<int>& c) {
13     int n = c.size();
14     vector<int> p(n);
15     Fenwick<AddMono<int>> fen(n);
16     for (int i = 0; i < n; i++) {
17         fen.add(i, 1);
18     }
19     for (int i = 0; i < n; i++) {
20         p[i] = fen.kth(c[i]);
21         fen.add(p[i], -1);
22     }
23     return p;
24 }
25
26 i64 toRank(vector<int>& p) {
27     int n = p.size();
28     auto c = toLehmer(p);
29     i64 fac = 1, rk = 0;
30     for (int i = n - 1; i >= 0; i--) {
31         rk += c[i] * fac;
32         fac *= (n - i);
33     }
34     return rk;
35 }
36
37 vector<int> fromRank(int n, i64 rk) {
38     vector<int> c(n);
39     for (int i = 1; i <= n; i++) {
40         c[n - i] = rk % i;
41         rk /= i;
42     }
43     return fromLehmer(c);
44 }

```

5.3 逆序对

```

1 i64 invCount(vector<i64>& a, int l, int r, vector<i64>& tmp) {
2     if (r - l <= 1) return 0;
3
4     i64 res = 0;
5     int mid = (l + r) / 2;
6     res += invCount(a, l, mid, tmp);

```

```

7   res += invCount(a, mid, r, tmp);
8
9   int i = 1, j = mid, k = 1;
10  while (i < mid && j < r) {
11      if (a[i] <= a[j]) {
12          tmp[k++] = a[i++];
13      } else {
14          tmp[k++] = a[j++];
15          res += mid - i;
16      }
17  }
18  while (i < mid) {
19      tmp[k++] = a[i++];
20  }
21  while (j < r) {
22      tmp[k++] = a[j++];
23  }
24
25  for (i = 1; i < r; i++) {
26      a[i] = tmp[i];
27  }
28
29  return res;
30 }
31

```

5.4 容斥原理

```

1  // cap(S) 表示 S 集合的交集大小
2
3  // 交集形式
4  template <typename T, typename F>
5  T ie(int n, F&& cap) {
6      T ans{};
7      for (int S = 1; S < (1 << n); S++) {
8          T cur = cap(S);
9          if (__builtin_popcount(S) & 1) {
10             ans += cur;
11         } else {
12             ans -= cur;
13         }
14     }
15     return ans;
16 }

```

```

17
18 // 补集形式
19 template <typename T, typename F>
20 T ieComp(int n, F&& cap) {
21     T ans{};
22     for (int S = 0; S < (1 << n); S++) {
23         T cur = cap(S);
24         if (__builtin_popcount(S) & 1) {
25             ans -= cur;
26         } else {
27             ans += cur;
28         }
29     }
30     return ans;
31 }
32
33 // 恰好满足 k 个性质的元素数量
34 template <typename T, typename F>
35 vector<T> ieExact(int n, F&& cap) {
36     vector<T> N(n + 1, T{});
37     for (int S = 0; S < (1 << n); S++) {
38         int k = __builtin_popcount(S);
39         N[k] += cap(S);
40     }
41
42     vector<T> binom(n + 2, T{}), E(n + 1, T{});
43     binom[0] = T{1};
44     for (int j = 0; j <= n; j++) {
45         for (int k = 0; k <= j; k++) {
46             T term = N[j] * binom[k];
47             if ((j - k) & 1) {
48                 E[k] -= term;
49             } else {
50                 E[k] += term;
51             }
52         }
53         for (int k = j + 1; k >= 1; k--) {
54             binom[k] += binom[k - 1];
55         }
56     }
57     return E;
58 }
59

```

```

60 // 至少满足 k 个
61 template <typename T, typename F>
62 vector<T> ieLeast(int n, F&& cap) {
63     auto E = ieExact<T>(n, cap);
64     for (int k = n - 1; k >= 0; k--) {
65         E[k] += E[k + 1];
66     }
67     return E;
68 }
69
70 // 至多满足 k 个
71 template <typename T, typename F>
72 vector<T> ieMost(int n, F&& cap) {
73     auto E = ieExact<T>(n, cap);
74     for (int k = 0; k < n; k++) {
75         E[k + 1] += E[k];
76     }
77     return E;
78 }

```

5.5 zeta 变换

```

1 // 子集和:  $f[\text{mask}] = \sum_{\text{sub} \subseteq \text{mask}} f[\text{sub}]$ 
2 // 方向: 小的 mask  $\rightarrow$  大的 mask ("吸收"不带位)
3 template <typename T>
4 void subsetZeta(vector<T>& f) {
5     int n = f.size();
6     for (int i = 1; i < n; i <= 1) {
7         for (int mask = 0; mask < n; mask++) {
8             if (mask & i) {
9                 f[mask] += f[mask ^ i];
10            }
11        }
12    }
13 }
14
15 // 子集和的逆
16 template <typename T>
17 void subsetMobius(vector<T>& f) {
18     int n = f.size();
19     for (int i = 1; i < n; i <= 1) {
20         for (int mask = 0; mask < n; mask++) {
21             if (mask & i) {

```

```

22         f[mask] -= f[mask ^ i];
23     }
24 }
25 }
26 }
27
28 // 超集和:  $f[\text{mask}] = \sum_{\text{sup} \supseteq \text{mask}} f[\text{sup}]$ 
29 // 方向: 大的 mask  $\rightarrow$  小的 mask ("分发"给不带这一位的 mask)
30 template <typename T>
31 void supersetZeta(vector<T>& f) {
32     int n = f.size();
33     for (int i = 1; i < n; i <= 1) {
34         for (int mask = 0; mask < n; mask++) {
35             if (mask & i) {
36                 f[mask ^ i] += f[mask];
37             }
38         }
39     }
40 }
41
42 // 超集和的逆
43 template <typename T>
44 void supersetMobius(vector<T>& f) {
45     int n = f.size();
46     for (int i = 1; i < n; i <= 1) {
47         for (int mask = 0; mask < n; mask++) {
48             if (mask & i) {
49                 f[mask ^ i] -= f[mask];
50             }
51         }
52     }
53 }

```

6 数据结构

6.1 线段树

6.1.1 线段树

```

1 template <typename Mono>
2 struct SegmentTree {
3     using X = typename Mono::Type;
4
5     int n;
6     vector<X> info;

```

```

7 SegmentTree() : n(0) {}
8 SegmentTree(const vector<X>& init_) {
9     init(init_);
10 }
11
12 void init(const vector<X>& init_) {
13     n = init_.size();
14     int logn = __lg(n);
15     info.assign(4 << logn, Mono::e());
16
17     auto build = [&](auto&& self, int p, int l, int r) -> void {
18         if (r - l == 1) {
19             info[p] = init_[l];
20             return;
21         }
22         int mid = (l + r) / 2;
23         self(self, 2 * p, l, mid);
24         self(self, 2 * p + 1, mid, r);
25         pull(p);
26     };
27
28     build(build, 1, 0, n);
29 }
30
31 void pull(int p) {
32     info[p] = Mono::op(info[2 * p], info[2 * p + 1]);
33 }
34
35 void modify(int p, int l, int r, int x, const X& v) {
36     if (r - l == 1) {
37         info[p] = v;
38         return;
39     }
40     int mid = (l + r) / 2;
41     if (x < mid) {
42         modify(2 * p, l, mid, x, v);
43     } else {
44         modify(2 * p + 1, mid, r, x, v);
45     }
46     pull(p);
47 }
48
49 void modify(int p, const X& v) {

```

```

50     modify(1, 0, n, p, v);
51 }
52
53 X rangeQuery(int p, int l, int r, int x, int y) {
54     if (y <= l || r <= x) {
55         return Mono::e();
56     }
57     if (x <= l && r <= y) {
58         return info[p];
59     }
60     int mid = (l + r) / 2;
61     return Mono::op(rangeQuery(2 * p, l, mid, x, y), rangeQuery(2 * p + 1, mid,
r, x, y));
62 }
63 X rangeQuery(int l, int r) {
64     return rangeQuery(1, 0, n, l, r);
65 }
66
67 template <typename F>
68 int findFirst(int p, int l, int r, int x, int y, F&& pred) {
69     if (y <= l || r <= x) {
70         return -1;
71     }
72     if (x <= l && r <= y && !pred(info[p])) {
73         return -1;
74     }
75     if (r - l == 1) {
76         return l;
77     }
78     int mid = (l + r) / 2;
79     int res = findFirst(2 * p, l, mid, x, y, pred);
80     if (res == -1) {
81         res = findFirst(2 * p + 1, mid, r, x, y, pred);
82     }
83     return res;
84 }
85
86 template <typename F>
87 int findFirst(int l, int r, F&& pred) {
88     return findFirst(1, 0, n, l, r, pred);
89 }
90
91 template <typename F>
92 int findLast(int p, int l, int r, int x, int y, F&& pred) {

```

```

92     if (y <= 1 || r <= x) {
93         return -1;
94     }
95     if (x <= 1 && r <= y && !pred(info[p])) {
96         return -1;
97     }
98     if (r - 1 == 1) {
99         return 1;
100    }
101    int mid = (1 + r) / 2;
102    int res = findLast(2 * p + 1, mid, r, x, y, pred);
103    if (res == -1) {
104        res = findLast(2 * p, 1, mid, x, y, pred);
105    }
106    return res;
107 }
108 template <typename F>
109 int findLast(int l, int r, F&& pred) {
110     return findLast(1, 0, n, l, r, pred);
111 }
112 };

```

6.1.2 懒标记线段树

```

1  template <typename ActMono>
2  struct LazySegmentTree {
3      using Mono = typename ActMono::Mono;
4      using Tag = typename ActMono::Tag;
5      using X = typename ActMono::X;
6      using Y = typename ActMono::Y;
7
8      int n;
9      vector<X> info;
10     vector<Y> tag;
11
12     LazySegmentTree() : n(0) {}
13     LazySegmentTree(const vector<X>& init_) {
14         init(init_);
15     }
16
17     void init(const vector<X>& init_) {
18         n = init_.size();
19         int logn = __lg(n);
20         info.assign(4 << logn, Mono::e());

```

```

21         tag.assign(4 << logn, Tag::e());
22
23         auto build = [&](auto&& self, int p, int l, int r) -> void {
24             if (r - 1 == 1) {
25                 info[p] = init_[l];
26                 return;
27             }
28             int mid = (1 + r) / 2;
29             self(self, 2 * p, l, mid);
30             self(self, 2 * p + 1, mid, r);
31             pull(p);
32         };
33
34         build(build, 1, 0, n);
35     }
36
37     void pull(int p) {
38         info[p] = Mono::op(info[2 * p], info[2 * p + 1]);
39     }
40
41     void apply(int p, int len, const Y& v) {
42         info[p] = ActMono::act(info[p], v, len);
43         tag[p] = Tag::op(tag[p], v);
44     }
45
46     void push(int p, int len) {
47         int llen = len / 2;
48         apply(2 * p, llen, tag[p]);
49         apply(2 * p + 1, len - llen, tag[p]);
50         tag[p] = Tag::e();
51     }
52
53     void modify(int p, int l, int r, int x, const X& v) {
54         if (r - 1 == 1) {
55             info[p] = v;
56             return;
57         }
58         int mid = (1 + r) / 2;
59         push(p, r - 1);
60         if (x < mid) {
61             modify(2 * p, l, mid, x, v);
62         } else {
63             modify(2 * p + 1, mid, r, x, v);
64         }

```



```

65     pull(p);
66 }
67 void modify(int p, const X& v) {
68     modify(1, 0, n, p, v);
69 }
70
71 X rangeQuery(int p, int l, int r, int x, int y) {
72     if (y <= 1 || r <= x) {
73         return Mono::e();
74     }
75     if (x <= 1 && r <= y) {
76         return info[p];
77     }
78     int mid = (1 + r) / 2;
79     push(p, r - 1);
80     return Mono::op(rangeQuery(2 * p, l, mid, x, y), rangeQuery(2 * p + 1, mid,
81 r, x, y));
82 }
83 X rangeQuery(int l, int r) {
84     return rangeQuery(1, 0, n, l, r);
85 }
86
87 void rangeApply(int p, int l, int r, int x, int y, const Y& v) {
88     if (y <= 1 || r <= x) {
89         return;
90     }
91     if (x <= 1 && r <= y) {
92         apply(p, r - 1, v);
93         return;
94     }
95     int mid = (1 + r) / 2;
96     push(p, r - 1);
97     rangeApply(2 * p, l, mid, x, y, v);
98     rangeApply(2 * p + 1, mid, r, x, y, v);
99     pull(p);
100 }
101 void rangeApply(int l, int r, const Y& v) {
102     return rangeApply(1, 0, n, l, r, v);
103 }
104
105 template <typename F>
106 int findFirst(int p, int l, int r, int x, int y, F&& pred) {

```

```

107     return -1;
108 }
109 if (x <= 1 && r <= y && !pred(info[p])) {
110     return -1;
111 }
112 if (r - 1 == 1) {
113     return 1;
114 }
115 int mid = (1 + r) / 2;
116 push(p, r - 1);
117 int res = findFirst(2 * p, l, mid, x, y, pred);
118 if (res == -1) {
119     res = findFirst(2 * p + 1, mid, r, x, y, pred);
120 }
121 return res;
122 }
123 template <typename F>
124 int findFirst(int l, int r, F&& pred) {
125     return findFirst(1, 0, n, l, r, pred);
126 }
127
128 template <typename F>
129 int findLast(int p, int l, int r, int x, int y, F&& pred) {
130     if (y <= 1 || r <= x) {
131         return -1;
132     }
133     if (x <= 1 && r <= y && !pred(info[p])) {
134         return -1;
135     }
136     if (r - 1 == 1) {
137         return 1;
138     }
139     int mid = (1 + r) / 2;
140     push(p, r - 1);
141     int res = findLast(2 * p + 1, mid, r, x, y, pred);
142     if (res == -1) {
143         res = findLast(2 * p, l, mid, x, y, pred);
144     }
145     return res;
146 }
147 template <typename F>
148 int findLast(int l, int r, F&& pred) {
149     return findLast(1, 0, n, l, r, pred);

```

```

150 }
151 };

```

6.1.3 动态开点线段树

```

1 template <typename ActMono>
2 struct DynamicSegmentTree {
3     using Mono = typename ActMono::Mono;
4     using Tag = typename ActMono::Tag;
5     using X = typename ActMono::X;
6     using Y = typename ActMono::Y;
7
8     int n;
9     int root;
10    vector<X> info;
11    vector<Y> tag;
12    vector<int> lc, rc;
13
14    DynamicSegmentTree() : n(0), root(-1) {}
15    DynamicSegmentTree(int n_) {
16        init(n_);
17    }
18
19    void init(int n_) {
20        n = n_;
21        info.clear();
22        tag.clear();
23        lc.clear();
24        rc.clear();
25        root = newNode();
26    }
27
28    int newNode() {
29        info.push_back(Mono::e());
30        tag.push_back(Tag::e());
31        lc.push_back(-1);
32        rc.push_back(-1);
33        return int(info.size()) - 1;
34    }
35
36    void pull(int p) {
37        X left = (lc[p] != -1 ? info[lc[p]] : Mono::e());
38        X right = (rc[p] != -1 ? info[rc[p]] : Mono::e());
39        info[p] = Mono::op(left, right);

```

```

40    }
41
42    void apply(int p, int len, const Y& v) {
43        info[p] = ActMono::act(info[p], v, len);
44        tag[p] = Tag::op(tag[p], v);
45    }
46
47    void push(int p, int len) {
48        if (lc[p] == -1) {
49            lc[p] = newNode();
50        }
51        if (rc[p] == -1) {
52            rc[p] = newNode();
53        }
54        int llen = len / 2;
55        apply(lc[p], llen, tag[p]);
56        apply(rc[p], len - llen, tag[p]);
57        tag[p] = Tag::e();
58    }
59
60    void modify(int p, int l, int r, int x, const X& v) {
61        if (r - l == 1) {
62            info[p] = v;
63            return;
64        }
65        int mid = (l + r) / 2;
66        push(p, r - l);
67        if (x < mid) {
68            modify(lc[p], l, mid, x, v);
69        } else {
70            modify(rc[p], mid, r, x, v);
71        }
72        pull(p);
73    }
74
75    void modify(int x, const X& v) {
76        modify(root, 0, n, x, v);
77    }
78
79    X rangeQuery(int p, int l, int r, int x, int y) {
80        if (p == -1 || y <= l || r <= x) {
81            return Mono::e();
82        }
83        if (x <= l && r <= y) {

```

```

83     return info[p];
84 }
85 int mid = (l + r) / 2;
86 push(p, r - 1);
87 return Mono::op(rangeQuery(lc[p], l, mid, x, y), rangeQuery(rc[p], mid, r,
x, y));
88 }
89 X rangeQuery(int l, int r) {
90     return rangeQuery(root, 0, n, l, r);
91 }
92
93 void rangeApply(int p, int l, int r, int x, int y, const Y& v) {
94     if (y <= l || r <= x) {
95         return;
96     }
97     if (x <= l && r <= y) {
98         apply(p, r - 1, v);
99         return;
100     }
101     int mid = (l + r) / 2;
102     push(p, r - 1);
103     rangeApply(lc[p], l, mid, x, y, v);
104     rangeApply(rc[p], mid, r, x, y, v);
105     pull(p);
106 }
107 void rangeApply(int l, int r, const Y& v) {
108     rangeApply(root, 0, n, l, r, v);
109 }
110
111 template <typename F>
112 int findFirst(int p, int l, int r, int x, int y, F&& pred) {
113     if (p == -1 || y <= l || r <= x) {
114         return -1;
115     }
116     if (x <= l && r <= y && !pred(info[p])) {
117         return -1;
118     }
119     if (r - l == 1) {
120         return l;
121     }
122     int mid = (l + r) / 2;
123     push(p, r - 1);
124     int res = findFirst(lc[p], l, mid, x, y, pred);

```

```

125     if (res == -1) {
126         res = findFirst(rc[p], mid, r, x, y, pred);
127     }
128     return res;
129 }
130 template <typename F>
131 int findFirst(int l, int r, F&& pred) {
132     return findFirst(root, 0, n, l, r, pred);
133 }
134
135 template <typename F>
136 int findLast(int p, int l, int r, int x, int y, F&& pred) {
137     if (p == -1 || y <= l || r <= x) {
138         return -1;
139     }
140     if (x <= l && r <= y && !pred(info[p])) {
141         return -1;
142     }
143     if (r - l == 1) {
144         return l;
145     }
146     int mid = (l + r) / 2;
147     push(p, r - 1);
148     int res = findLast(rc[p], mid, r, x, y, pred);
149     if (res == -1) {
150         res = findLast(lc[p], l, mid, x, y, pred);
151     }
152     return res;
153 }
154 template <typename F>
155 int findLast(int l, int r, F&& pred) {
156     return findLast(root, 0, n, l, r, pred);
157 }
158 };

```

6.2 树状数组

6.2.1 树状数组

```

1 template <typename Mono>
2 struct Fenwick {
3     using X = typename Mono::Type;
4
5     int n;
6     vector<X> a;

```

```

7
8 Fenwick() : n(0) {}
9 Fenwick(int n_) {
10     init(n_);
11 }
12
13 void init(int n_) {
14     n = n_;
15     a.assign(n, Mono::e());
16 }
17
18 void add(int x, const X& v) {
19     assert(0 <= x && x < n);
20     for (int i = x + 1; i <= n; i += i & -i) {
21         a[i - 1] = Mono::op(a[i - 1], v);
22     }
23 }
24
25 X sum(int x) {
26     assert(0 <= x && x <= n);
27     X res = Mono::e();
28     for (int i = x; i > 0; i -= i & -i) {
29         res = Mono::op(a[i - 1], res);
30     }
31     return res;
32 }
33
34 X rangeSum(int l, int r) {
35     assert(l <= r);
36     return Mono::op(Mono::inv(sum(l)), sum(r));
37 }
38
39 // lower_bound
40 int kth(X k) {
41     int x = 0;
42     for (int i = 1 << __lg(n); i; i /= 2) {
43         if (x + i <= n && k >= a[x + i - 1]) {
44             x += i;
45             k -= a[x - 1];
46         }
47     }
48     return x;
49 }

```

```

50 };

```

6.2.2 区间加、单点查树状数组

```

1 template <typename Mono>
2 struct DualFenwick {
3     using X = typename Mono::Type;
4
5     int n;
6     Fenwick<Mono> fen;
7
8     DualFenwick() : n(0), fen(0) {}
9     DualFenwick(int n_) {
10         init(n_);
11     }
12
13     void init(int n_) {
14         n = n_;
15         fen.init(n);
16     }
17
18     void add(int l, int r, const X& v) {
19         assert(0 <= l && l <= r && r <= n);
20         fen.add(l, v);
21         if (r < n) {
22             fen.add(r, Mono::inv(v));
23         }
24     }
25
26     X query(int x) {
27         assert(0 <= x && x < n);
28         return fen.sum(x + 1);
29     }
30 };

```

6.2.3 区间加、区间和树状数组

```

1 template <typename Mono>
2 struct FenwickRangeAdd {
3     using X = typename Mono::Type;
4
5     int n;
6     Fenwick<Mono> fen0, fen1;
7
8     FenwickRangeAdd() : n(0), fen0(1), fen1(1) {}
9     FenwickRangeAdd(int n_) {

```

```

10     init(n_);
11 }
12
13 void init(int n_) {
14     n = n_;
15     fen0.init(n + 1);
16     fen1.init(n + 1);
17 }
18
19 void add(int l, int r, const X& v) {
20     assert(0 <= l && l <= r && r <= n);
21     fen0.add(l, Mono::power(v, -static_cast<i64>(l)));
22     fen0.add(r, Mono::power(v, static_cast<i64>(r)));
23     fen1.add(l, v);
24     fen1.add(r, Mono::inv(v));
25 }
26
27 X sum(int r) {
28     assert(0 <= r && r <= n);
29     return Mono::op(Mono::power(fen1.sum(r), static_cast<i64>(r)),
30 fen0.sum(r));
31 }
32
33 X rangeSum(int l, int r) {
34     assert(0 <= l && l <= r && r <= n);
35     return Mono::op(Mono::inv(sum(l)), sum(r));
36 }
37 };

```

6.2.4 二维树状数组

```

1 template <typename Mono>
2 struct Fenwick2D {
3     using X = typename Mono::Type;
4
5     int h, w;
6     vector<X> a;
7
8     Fenwick2D() : h(0), w(0) {}
9     Fenwick2D(int h_, int w_) {
10         init(h_, w_);
11     }
12
13     void init(int h_, int w_) {
14         h = h_;

```

```

15         w = w_;
16         a.assign((h + 1) * (w + 1), Mono::e());
17     }
18
19     void add(int x, int y, const X& v) {
20         assert(0 <= x && x < h);
21         assert(0 <= y && y < w);
22         for (int i = x + 1; i <= h; i += i & -i) {
23             for (int j = y + 1; j <= w; j += j & -j) {
24                 a[i * (w + 1) + j] = Mono::op(a[i * (w + 1) + j], v);
25             }
26         }
27     }
28
29     X sum(int x, int y) {
30         assert(0 <= x && x <= h);
31         assert(0 <= y && y <= w);
32         X res = Mono::e();
33         for (int i = x; i > 0; i -= i & -i) {
34             for (int j = y; j > 0; j -= j & -j) {
35                 res = Mono::op(res, a[i * (w + 1) + j]);
36             }
37         }
38         return res;
39     }
40
41     X rangeSum(int x1, int x2, int y1, int y2) {
42         assert(0 <= x1 && x1 <= x2 && x2 <= h);
43         assert(0 <= y1 && y1 <= y2 && y2 <= w);
44         X s22 = sum(x2, y2);
45         X s12 = sum(x1, y2);
46         X s21 = sum(x2, y1);
47         X s11 = sum(x1, y1);
48         return Mono::op(Mono::op(s22, Mono::inv(s12)), Mono::op(Mono::inv(s21),
49 s11));
50     }
51 };

```

6.3 并查集

```

1 struct DSU {
2     vector<int> f, siz;
3
4     DSU() {}

```

```

5  DSU(int n) {
6      init(n);
7  }
8
9  void init(int n) {
10     f.resize(n);
11     iota(f.begin(), f.end(), 0);
12     siz.assign(n, 1);
13 }
14
15 int find(int x) {
16     while (x != f[x]) {
17         x = f[x] = f[f[x]];
18     }
19     return x;
20 }
21
22 bool same(int x, int y) {
23     return find(x) == find(y);
24 }
25
26 bool merge(int x, int y) {
27     x = find(x);
28     y = find(y);
29     if (x == y) {
30         return false;
31     }
32     siz[x] += siz[y];
33     f[y] = x;
34     return true;
35 }
36
37 int size(int x) {
38     return siz[find(x)];
39 }
40 };

```

6.4 稀疏表

```

1  template <typename Mono>
2  struct SparseTable {
3      using X = typename Mono::Type;
4
5      int n;
6      vector<vector<X>> st;

```

```

7
8      SparseTable(const vector<X>& a) {
9          init(a);
10     }
11
12     void init(const vector<X>& a) {
13         n = a.size();
14         int logn = __lg(n);
15         st.assign(logn + 1, vector<X>(n));
16         for (int i = 0; i < n; i++) {
17             st[0][i] = a[i];
18         }
19         for (int j = 1; (1 << j) <= n; j++) {
20             for (int i = 0; i + (1 << j) <= n; i++) {
21                 st[j][i] = Mono::op(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
22             }
23         }
24     }
25
26     X query(int l, int r) {
27         assert(l < r);
28         int len = r - l;
29         int t = __lg(len);
30         return Mono::op(st[t][l], st[t][r - (1 << t)]);
31     }
32 };

```

6.5 01-Trie

```

1  struct BinaryTrie {
2      struct Node {
3          array<int, 2> ch;
4          int end;
5          Node() {
6              ch[0] = ch[1] = 0;
7              end = 0;
8          }
9      };
10
11      vector<Node> tr;
12      int tot;
13
14      BinaryTrie(int n) {
15          init(n);

```

```

16 }
17
18 void init(int n) {
19     tr.resize(n);
20     tot = 1;
21 }
22
23 void insert(int x) {
24     int p = 1;
25     for (int i = 30; i >= 0; i--) {
26         int c = x >> i & 1;
27         if (!tr[p].ch[c]) {
28             ++tot;
29             tr[p].ch[c] = tot;
30         }
31         p = tr[p].ch[c];
32         tr[p].end++;
33     }
34 }
35
36 int query(int x) {
37     int p = 1, res = 0;
38     for (int i = 30; i >= 0; i--) {
39         int c = x >> i & 1;
40         if (tr[p].ch[c ^ 1]) {
41             res |= (1 << i);
42             p = tr[p].ch[c ^ 1];
43         } else {
44             p = tr[p].ch[c];
45         }
46     }
47     return res;
48 }
49
50 void erase(int x) {
51     int p = 1;
52     for (int i = 30; i >= 0; i--) {
53         int c = x >> i & 1;
54         int nxt = tr[p].ch[c];
55         tr[nxt].end--;
56         if (tr[nxt].end == 0) {
57             tr[p].ch[c] = 0;
58         }
59     }
60 }

```

```

59     p = nxt;
60 }
61 }
62 };
63

```

6.6 笛卡尔树

```

1 template <typename T, bool IS_MIN = false>
2 struct CartesianTree {
3     int n;
4     vector<T>& a;
5     vector<int> L, R, p;
6     vector<pair<int, int>> range;
7     int root;
8
9     CartesianTree(vector<T>& a_) : n(a_.size()), a(a_), root(-1) {
10         L.assign(n, -1);
11         R.assign(n, -1);
12         p.assign(n, -1);
13         range.assign(n, {0, n});
14         vector<int> stk;
15
16         auto cmp = [&](int i, int j) -> bool {
17             if (a[i] == a[j]) {
18                 return i < j;
19             }
20             if constexpr (IS_MIN) {
21                 return a[i] < a[j];
22             } else {
23                 return a[j] < a[i];
24             }
25         };
26
27         for (int i = 0; i < n; i++) {
28             while (!stk.empty() && cmp(i, stk.back())) {
29                 L[i] = stk.back();
30                 stk.pop_back();
31             }
32             range[i].first = stk.empty() ? 0 : stk.back() + 1;
33             if (!stk.empty()) {
34                 R[stk.back()] = i;
35             }
36             stk.push_back(i);
37         }
38     }
39 };

```

```

37     }
38     if (!stk.empty()) {
39         root = stk.front();
40     }
41     stk.clear();
42     for (int i = n - 1; i >= 0; i--) {
43         while (!stk.empty() && cmp(i, stk.back())) {
44             stk.pop_back();
45         }
46         range[i].second = stk.empty() ? n : stk.back();
47         stk.push_back(i);
48     }
49     for (int i = 0; i < n; i++) {
50         if (L[i] != -1) {
51             p[L[i]] = i;
52         }
53         if (R[i] != -1) {
54             p[R[i]] = i;
55         }
56     }
57 }
58 };

```

7 线性代数

7.1 矩阵

```

1 template <typename T>
2 struct Matrix {
3     int n, m;
4     vector<vector<T>>> a;
5
6     Matrix() : n(0), m(0) {}
7     Matrix(int n_, int m_, T v = T{}) : n(n_), m(m_), a(n, vector<T>(m, v)) {}
8     Matrix(int n_) : Matrix(n_, n_) {}
9
10    static Matrix identity(int n) {
11        Matrix res(n);
12        for (int i = 0; i < n; i++) {
13            res[i][i] = T{1};
14        }
15        return res;
16    }
17 }

```

```

18 vector<T>& operator[](int i) {
19     return a[i];
20 }
21 const vector<T>& operator[](int i) const {
22     return a[i];
23 }
24 };
25
26 template <typename T>
27 Matrix<T> operator*(const Matrix<T>& a, const Matrix<T>& b) {
28     assert(a.m == b.n);
29     Matrix<T> c(a.n, b.m);
30     for (int i = 0; i < c.n; i++) {
31         for (int k = 0; k < a.m; k++) {
32             for (int j = 0; j < c.m; j++) {
33                 c.a[i][j] += a.a[i][k] * b.a[k][j];
34             }
35         }
36     }
37     return c;
38 }
39
40 template <typename T>
41 Matrix<T>& operator*=(Matrix<T>& a, const Matrix<T>& b) {
42     return a = a * b;
43 }
44
45 template <typename T, integral U>
46 Matrix<T> power(Matrix<T> a, U b) {
47     Matrix<T> res = Matrix<T>::identity(a.n);
48     for (; b > 0; b >>= 1) {
49         if (b & 1) {
50             res *= a;
51         }
52         a *= a;
53     }
54     return res;
55 }
56
57 template <typename T>
58 Matrix<T> transpose(const Matrix<T>& a) {
59     Matrix<T> res(a.m, a.n);
60     for (int i = 0; i < a.n; i++) {

```



```

61     for (int j = 0; j < a.m; j++) {
62         res[j][i] = a[i][j];
63     }
64 }
65 return res;
66 }

```

7.2 高斯消元

```

1 template <typename T>
2 pair<vector<int>, T> gauss(Matrix<T>& a) {
3     vector<int> pivot;
4     T prod = T{1};
5     int r = 0;
6     for (int j = 0; j < a.m && r < a.n; j++) {
7         int p = r;
8         while (p < a.n && a[p][j] == T{0}) {
9             p++;
10        }
11        if (p == a.n) {
12            continue;
13        }
14        if (p != r) {
15            swap(a[p], a[r]);
16            prod = -prod;
17        }
18        T x = a[r][j];
19        prod *= x;
20        pivot.push_back(j);
21        for (int i = r + 1; i < a.n; i++) {
22            if (a[i][j] == T{0}) {
23                continue;
24            }
25            T c = a[i][j] / x;
26            for (int k = j; k < a.m; k++) {
27                a[i][k] -= c * a[r][k];
28            }
29        }
30        r++;
31    }
32    return {pivot, prod};
33 }

```

7.3 矩阵秩

```

1 template <typename T>
2 int matrixRank(Matrix<T> a) {
3     auto [pivot, prod] = gauss(a);
4     return pivot.size();
5 }

```

7.4 逆矩阵

```

1 template <typename T>
2 pair<bool, Matrix<T>> matrixInv(Matrix<T> a) {
3     if (a.n != a.m) {
4         return {false, {}};
5     }
6
7     int n = a.n;
8     Matrix<T> b(n, 2 * n);
9
10    for (int i = 0; i < n; i++) {
11        for (int j = 0; j < n; j++) {
12            b[i][j] = a[i][j];
13        }
14        b[i][n + i] = T{1};
15    }
16
17    for (int col = 0; col < n; col++) {
18        int p = col;
19        while (p < n && b[p][col] == T{0}) {
20            p++;
21        }
22        if (p == n) {
23            return {false, {}};
24        }
25
26        swap(b[p], b[col]);
27
28        T x = b[col][col];
29        for (int j = 0; j < 2 * n; j++) {
30            b[col][j] /= x;
31        }
32
33        for (int i = 0; i < n; i++) {
34            if (i == col || b[i][col] == T{0}) {
35                continue;

```

```

36     }
37
38     T c = b[i][col];
39     for (int j = 0; j < 2 * n; j++) {
40         b[i][j] -= c * b[col][j];
41     }
42 }
43 }
44
45 Matrix<T> res(n, n);
46 for (int i = 0; i < n; i++) {
47     for (int j = 0; j < n; j++) {
48         res[i][j] = b[i][n + j];
49     }
50 }
51
52 return {true, res};
53 }

```

7.5 线性方程组

```

1 template <typename T>
2 vector<vector<T>> solveLinear(Matrix<T> a, vector<T> b) {
3     int n = a.n, m = a.m;
4     int r = 0;
5     vector<int> pivot(m, -1);
6
7     for (int j = 0; j < m && r < n; j++) {
8         int p = r;
9         while (p < n && a[p][j] == T{0}) {
10             p++;
11         }
12         if (p == n) {
13             continue;
14         }
15         swap(a[r], a[p]);
16         swap(b[r], b[p]);
17         T c = T{1} / a[r][j];
18         for (auto& x : a[r]) {
19             x *= c;
20         }
21         b[r] *= c;
22         for (int i = 0; i < n; i++) {
23             if (i == r || a[i][j] == T{0}) {

```

```

24             continue;
25         }
26         T c = a[i][j];
27         b[i] -= b[r] * c;
28         for (int k = j; k < m; k++) {
29             a[i][k] -= a[r][k] * c;
30         }
31     }
32     pivot[j] = r++;
33 }
34 for (int i = r; i < n; i++) {
35     if (b[i] != T{0}) {
36         return {};
37     }
38 }
39 vector<vector<T>> res(1, vector<T>(m));
40 for (int j = 0; j < m; j++) {
41     if (pivot[j] != -1) {
42         res[0][j] = b[pivot[j]];
43     }
44 }
45 for (int j = 0; j < m; j++) {
46     if (pivot[j] != -1) {
47         continue;
48     }
49     vector<T> x(m);
50     x[j] = T{-1};
51     for (int k = 0; k < j; k++) {
52         if (pivot[k] != -1) {
53             x[k] = a[pivot[k]][j];
54         }
55     }
56     res.push_back(x);
57 }
58
59 return res;
60 }

```

7.6 GF(2) 空间

7.6.1 线性基

```

1 struct Basis {
2     array<i64, 64> p{};
3 }

```

```

4  int size() {
5      int r = 0;
6      for (int i = 0; i < 64; i++) {
7          r += p[i] != 0;
8      }
9      return r;
10 }
11
12 bool insert(i64 x) {
13     for (int i = 63; i >= 0; i--) {
14         if (!(x >> i & 1)) {
15             continue;
16         }
17         if (!p[i]) {
18             p[i] = x;
19             return true;
20         }
21         x ^= p[i];
22     }
23     return false;
24 }
25
26 bool check(i64 x) {
27     for (int i = 63; i >= 0; i--) {
28         if (x >> i & 1) {
29             x ^= p[i];
30         }
31     }
32     return x == 0;
33 }
34
35 i64 getMax(i64 start = 0) {
36     i64 res = start;
37     for (int i = 63; i >= 0; i--) {
38         if ((res ^ p[i]) > res) {
39             res ^= p[i];
40         }
41     }
42     return res;
43 }
44
45 i64 getMin() {
46     for (int i = 0; i < 64; i++) {

```

```

47         if (p[i]) {
48             return p[i];
49         }
50     }
51     return 0;
52 }
53
54 void gauss() {
55     for (int i = 63; i >= 0; i--) {
56         if (!p[i]) {
57             continue;
58         }
59         for (int j = i - 1; j >= 0; j--) {
60             if (p[i] >> j & 1) {
61                 p[i] ^= p[j];
62             }
63         }
64     }
65 }
66
67 i64 kth(i64 k) {
68     vector<i64> b;
69     for (int i = 0; i < 64; i++) {
70         if (p[i]) {
71             b.push_back(p[i]);
72         }
73     }
74     int r = b.size();
75     if (k < 1 || (r < 63 && k > (1LL << r))) {
76         return -1;
77     }
78     k--;
79     i64 res = 0;
80     for (int i = 0; i < r; i++) {
81         if (k >> i & 1) {
82             res ^= b[i];
83         }
84     }
85     return res;
86 }
87 };

```

7.6.2 逆矩阵

```

1 namespace gf2 {
2
3 template <typename U>
4 vector<U> inverse(int n, vector<U> a) {
5     vector<U> b(n);
6     for (int i = 0; i < n; i++) {
7         b[i] = U{1} << i;
8     }
9     for (int j = 0; j < n; j++) {
10        int p = j;
11        while (p < n && !(a[p] >> j & U{1})) {
12            p++;
13        }
14        if (p == n) {
15            return {};
16        }
17        swap(a[p], a[j]);
18        swap(b[p], b[j]);
19        for (int i = 0; i < n; i++) {
20            if (i != j && (a[i] >> j & U{1})) {
21                a[i] ^= a[j];
22                b[i] ^= b[j];
23            }
24        }
25    }
26    return b;
27 }
28
29 }

```

7.6.3 转置

```

1 namespace gf2 {
2
3 template <typename U>
4 vector<U> transpose(int n, int m, const vector<U>& a) {
5     vector<U> b(m);
6     for (int i = 0; i < n; i++) {
7         for (int j = 0; j < m; j++) {
8             if (a[i] >> j & U{1}) {
9                 b[j] |= U{1} << i;
10            }
11        }
12    }
13 }

```

```

12 }
13     return b;
14 }
15
16 }

```

8 字符串

8.1 Manacher

```

1 vector<int> manacher(string s) {
2     string t = "#";
3     for (auto c : s) {
4         t += c;
5         t += '#';
6     }
7     int n = t.size();
8     vector<int> r(n);
9     for (int i = 0, j = 0; i < n; i++) {
10        if (2 * j - i >= 0 && j + r[j] > i) {
11            r[i] = min(r[2 * j - i], j + r[j] - i);
12        }
13        while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
14            r[i] += 1;
15        }
16        if (i + r[i] > j + r[j]) {
17            j = i;
18        }
19    }
20    return r;
21 }

```

8.2 KMP

```

1 vector<int> kmp(string s) {
2     int n = s.size();
3     vector<int> f(n + 1);
4     for (int i = 1, j = 0; i < n; i++) {
5         while (j && s[i] != s[j]) {
6             j = f[j];
7         }
8         j += (s[i] == s[j]);
9         f[i + 1] = j;
10    }
11    return f;
12 }

```

```
12 }
13
```

8.3 Trie

```
1 struct Trie {
2     struct Node {
3         array<int, 26> nxt;
4         int pass, end;
5
6         Node() {
7             nxt.fill(-1);
8             end = 0;
9             pass = 0;
10        }
11    };
12
13    vector<Node> tr;
14    int tot;
15
16    Trie(int n) {
17        init(n);
18    }
19
20    void init(int n) {
21        tr.resize(n);
22        tot = 0;
23    }
24
25    void insert(string& s) {
26        int p = 0;
27        ++tr[p].pass;
28        for (char& ch : s) {
29            int c = ch - 'a';
30            if (tr[p].nxt[c] == -1) {
31                ++tot;
32                tr[p].nxt[c] = tot;
33            }
34            p = tr[p].nxt[c];
35            ++tr[p].pass;
36        }
37        ++tr[p].end;
38    }
39
40    int count(string& s) {
```

```
41        int p = 0;
42        for (char& ch : s) {
43            int c = ch - 'a';
44            if (tr[p].nxt[c] == -1) {
45                return false;
46            }
47            p = tr[p].nxt[c];
48        }
49        return tr[p].end;
50    }
51 };
52
```

8.4 Z 函数

```
1 vector<int> Z(string s) {
2     int n = s.size();
3     vector<int> z(n + 1);
4     z[0] = n;
5     for (int i = 1, j = 1; i < n; i++) {
6         z[i] = max(0, min(j + z[j] - i, z[i - j]));
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
8             z[i]++;
9         }
10        if (i + z[i] > j + z[j]) {
11            j = i;
12        }
13    }
14    return z;
15 }
16
```

8.5 AC 自动机

```
1 struct AhoCorasick {
2     static constexpr int ALPHABET = 26;
3     struct Node {
4         int len;
5         int link;
6         array<int, ALPHABET> next;
7         Node() : len{0}, link{0}, next{} {}
8     };
9
10    vector<Node> t;
11
```

```

12 AhoCorasick() {
13     init();
14 }
15
16 void init() {
17     t.assign(2, Node());
18     t[0].next.fill(1);
19     t[0].len = -1;
20 }
21
22 int newNode() {
23     t.emplace_back();
24     return t.size() - 1;
25 }
26
27 int add(const string &a) {
28     int p = 1;
29     for (auto c : a) {
30         int x = c - 'a';
31         if (t[p].next[x] == 0) {
32             t[p].next[x] = newNode();
33             t[t[p].next[x]].len = t[p].len + 1;
34         }
35         p = t[p].next[x];
36     }
37     return p;
38 }
39
40 void work() {
41     queue<int> q;
42     q.push(1);
43
44     while (!q.empty()) {
45         int x = q.front();
46         q.pop();
47
48         for (int i = 0; i < ALPHABET; i++) {
49             if (t[x].next[i] == 0) {
50                 t[x].next[i] = t[t[x].link].next[i];
51             } else {
52                 t[t[x].next[i]].link = t[t[x].link].next[i];
53                 q.push(t[x].next[i]);
54             }
55         }
56     }
57 }
58
59 int next(int p, int x) {
60     return t[p].next[x];
61 }
62
63 int link(int p) {
64     return t[p].link;
65 }
66
67 int len(int p) {
68     return t[p].len;
69 }
70
71 int size() {
72     return t.size();
73 }
74 };
75

```

```

55     }
56 }
57 }
58
59 int next(int p, int x) {
60     return t[p].next[x];
61 }
62
63 int link(int p) {
64     return t[p].link;
65 }
66
67 int len(int p) {
68     return t[p].len;
69 }
70
71 int size() {
72     return t.size();
73 }
74 };
75

```

9 杂项

9.1 debug

```

1 #ifndef DEBUG_H
2 #define DEBUG_H
3
4
5 #define dbg(...) \
6     do { \
7         std::cerr << "[" << __FILE__ << ":" << __LINE__ << "]" "; \
8         dbg::print(__VA_ARGS__); \
9         std::cerr << std::endl; \
10    } while(0)
11
12 namespace dbg {
13
14 // 基本类型打印
15 template<typename T>
16 void print(const T& value) {
17     std::cerr << value;
18 }
19

```

```

20 // 字符特殊处理
21 inline void print(char c) {
22     if (c >= 32 && c <= 126)
23         std::cerr << "'" << c << "'";
24     else
25         std::cerr << "\\x" << std::hex << (int)c << "'";
26 }
27
28 // 字符串
29 inline void print(const std::string& s) {
30     std::cerr << "\"" << s << "\"";
31 }
32 inline void print(const char* s) {
33     std::cerr << "\"" << s << "\"";
34 }
35
36 // 多重参数
37 template<typename First, typename... Rest>
38 void print(const First& first, const Rest&... rest) {
39     print(first);
40     if (sizeof...(rest) > 0) {
41         std::cerr << " ";
42         print(rest...);
43     }
44 }
45
46 // vector
47 template<typename T>
48 void print(const std::vector<T>& v) {
49     std::cerr << "[";
50     for (size_t i = 0; i < v.size(); ++i) {
51         if (i > 0) std::cerr << ", ";
52         print(v[i]);
53     }
54     std::cerr << "]";
55 }
56
57 // map
58 template<typename K, typename V>
59 void print(const std::map<K, V>& m) {
60     std::cerr << "{";
61     size_t i = 0;
62     for (const auto& p : m) {

```

```

63         if (i++ > 0) std::cerr << ", ";
64         print(p.first);
65         std::cerr << ": ";
66         print(p.second);
67     }
68     std::cerr << "}";
69 }
70
71 // pair
72 template<typename T1, typename T2>
73 void print(const std::pair<T1, T2>& p) {
74     std::cerr << "(";
75     print(p.first);
76     std::cerr << ", ";
77     print(p.second);
78     std::cerr << ")";
79 }
80
81 // deque
82 template<typename T>
83 void print(const std::deque<T>& dq) {
84     std::cerr << "[";
85     for (size_t i = 0; i < dq.size(); ++i) {
86         if (i > 0) std::cerr << ", ";
87         print(dq[i]);
88     }
89     std::cerr << "]";
90 }
91
92 // set
93 template<typename T>
94 void print(const std::set<T>& s) {
95     std::cerr << "{";
96     size_t i = 0;
97     for (const auto& x : s) {
98         if (i++ > 0) std::cerr << ", ";
99         print(x);
100     }
101     std::cerr << "}";
102 }
103
104 // multiset
105 template<typename T>

```

```

106 void print(const std::multiset<T>& ms) {
107     std::cerr << "{";
108     size_t i = 0;
109     for (const auto& x : ms) {
110         if (i++ > 0) std::cerr << ", ";
111         print(x);
112     }
113     std::cerr << "}";
114 }
115
116 // queue
117 template<typename T>
118 void print(std::queue<T> q) {
119     std::cerr << "[";
120     bool first = true;
121     while (!q.empty()) {
122         if (!first) std::cerr << ", ";
123         first = false;
124         print(q.front());
125         q.pop();
126     }
127     std::cerr << "]";
128 }
129
130 // stack
131 template<typename T>
132 void print(std::stack<T> s) {
133     std::vector<T> temp;
134     while (!s.empty()) {
135         temp.push_back(s.top());
136         s.pop();
137     }
138     std::reverse(temp.begin(), temp.end());
139     print(temp);
140 }
141
142 // priority_queue
143 template<typename T>
144 void print(std::priority_queue<T> pq) {
145     std::vector<T> temp;
146     while (!pq.empty()) {
147         temp.push_back(pq.top());

```

```

149         pq.pop();
150     }
151     print(temp);
152 }
153
154 }
155 #endif
156

```

9.2 template

```

1 using namespace std;
2
3 using i64 = int64_t;
4 using u64 = uint64_t;
5 using u32 = uint32_t;
6
7 using i128 = __int128_t;
8 using u128 = __uint128_t;
9
10 template <typename T>
11 constexpr T inf = numeric_limits<T>::max() / 2;
12 template <>
13 constexpr int inf<int> = 1e9;
14 template <>
15 constexpr i64 inf<i64> = 1e18;
16
17 void solve();
18
19 int main() {
20     ios::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     int t;
24     cin >> t;
25
26     while (t--) {
27         solve();
28     }
29
30     return 0;
31 }

```

9.3 i128

```

1 using i128 = __int128;

```



```

2
3 ostream& operator<<(ostream& os, i128 n) {
4     if (n == 0) {
5         return os << 0;
6     }
7     string s;
8     while (n > 0) {
9         s += char('0' + n % 10);
10        n /= 10;
11    }
12    reverse(s.begin(), s.end());
13    return os << s;
14 }
15
16 i128 toi128(const string& s) {
17     i128 n = 0;
18     for (auto c : s) {
19         n = n * 10 + (c - '0');
20     }
21     return n;
22 }
23
24 i128 sqrti128(i128 n) {
25     i128 lo = 0, hi = 1E16;
26     while (lo < hi) {
27         i128 x = (lo + hi + 1) / 2;
28         if (x * x <= n) {
29             lo = x;
30         } else {
31             hi = x - 1;
32         }
33     }
34     return lo;
35 }
36
37 i128 gcd(i128 a, i128 b) {
38     while (b) {
39         a %= b;
40         swap(a, b);
41     }
42     return a;
43 }
44

```

9.4 日期

```

1 const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
2
3 bool isLeap(int y) {
4     return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
5 }
6
7 int daysInMonth(int y, int m) {
8     return d[m - 1] + (isLeap(y) && m == 2);
9 }
10
11 int getDay(int y, int m, int d) {
12     int ans = 0;
13     for (int i = 1970; i < y; i++) {
14         ans += 365 + isLeap(i);
15     }
16     for (int i = 1; i < m; i++) {
17         ans += daysInMonth(y, i);
18     }
19     ans += d;
20     return (ans + 2) % 7 + 1;
21 }
22

```